



life.augmented

STM32C0 MOOC Lab 1: Blinky

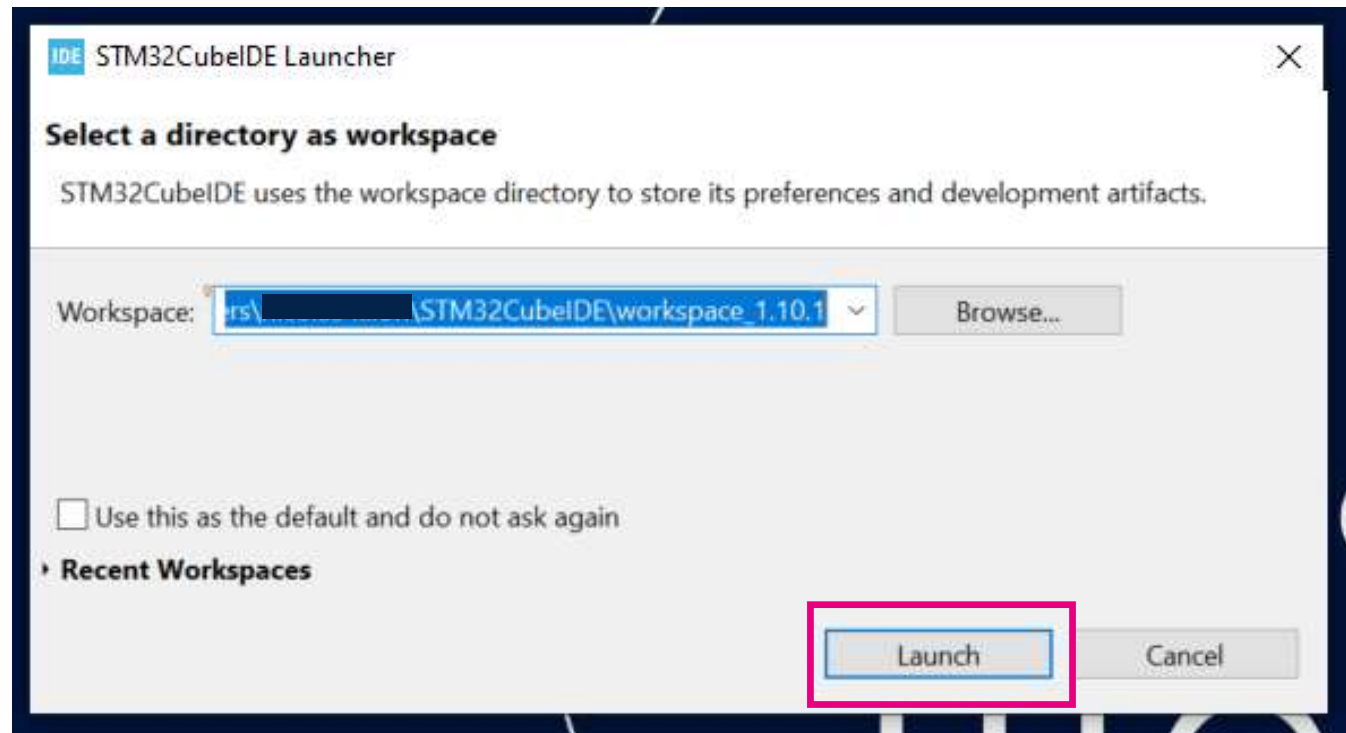
Nicolas FILLON

Lab: blinky

Objective:

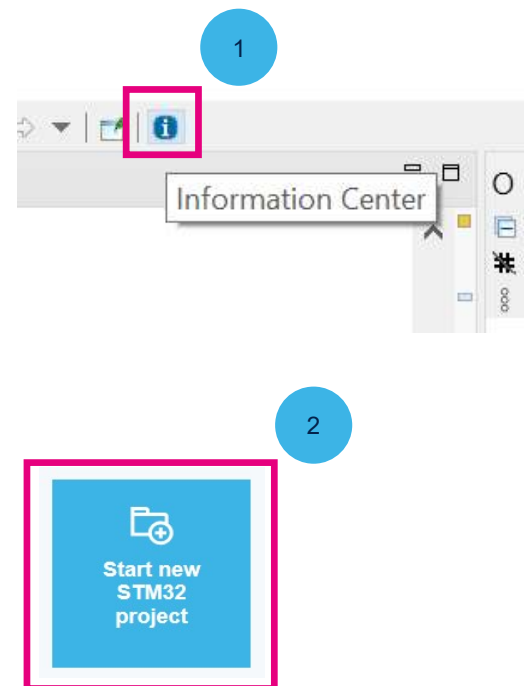
- The objective of this lab is to generate a simple project using STM32CubeIDE Software.
- In this example we are going to blink one of the LEDs present on the STM32C031 Nucleo board, connected to PA5 of the STM32C0 MCU.

Run STM32CubeIDE



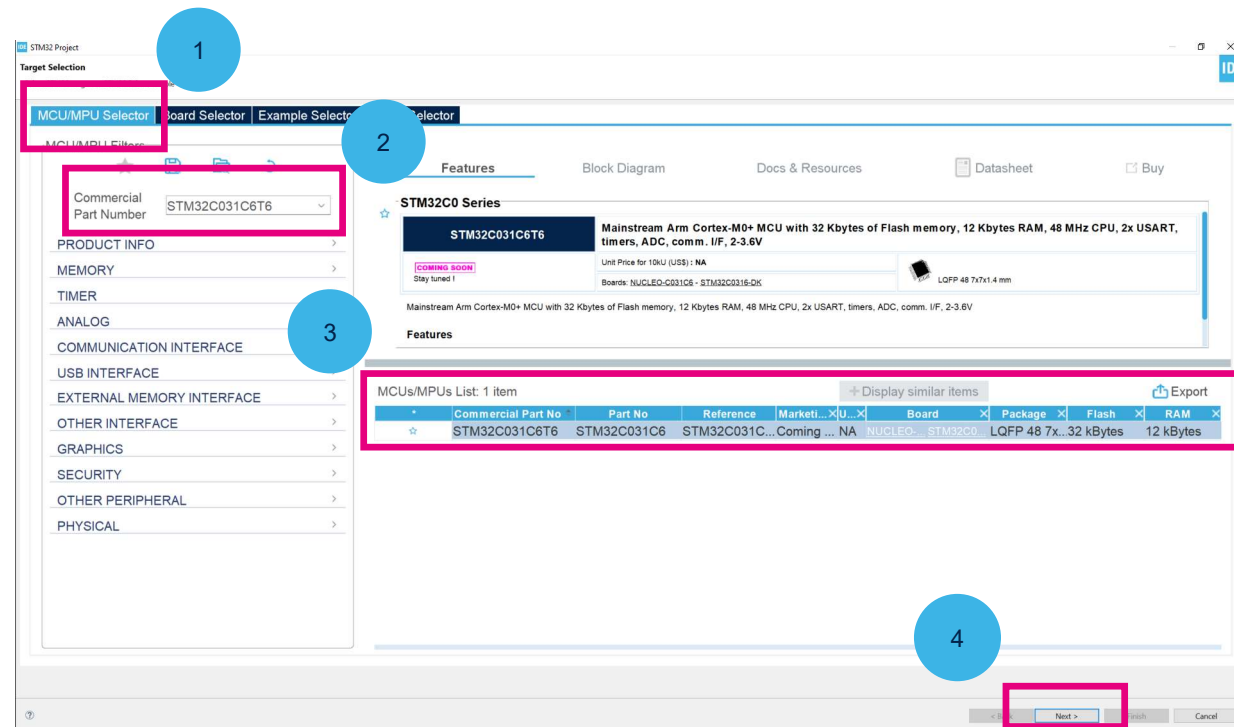
Create new project

- Click the Information Center icon 1
- Start new STM32 project 2

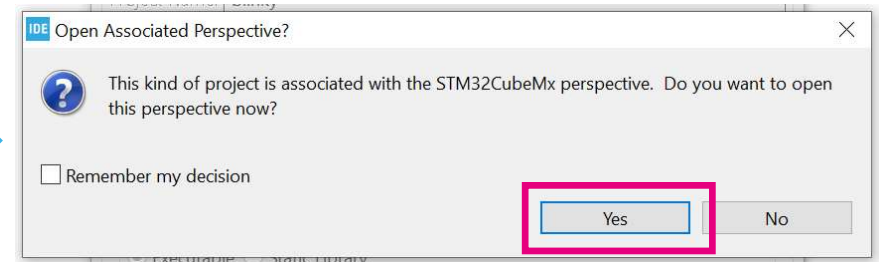
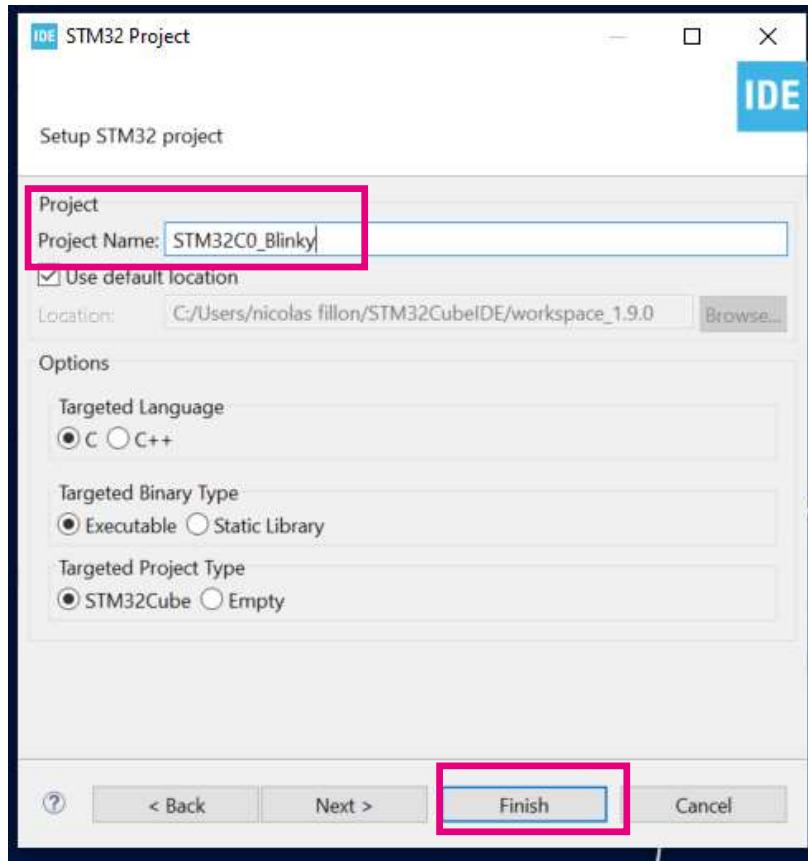


Create new project

- Click Access To MCU Selector 1
- Type: “STM32C031C6T6” in the Part Number Search 2
- Then Select STM32C031C6T6 3
 - LQFP48, 32 kB Flash 3
- Click “Next” 4

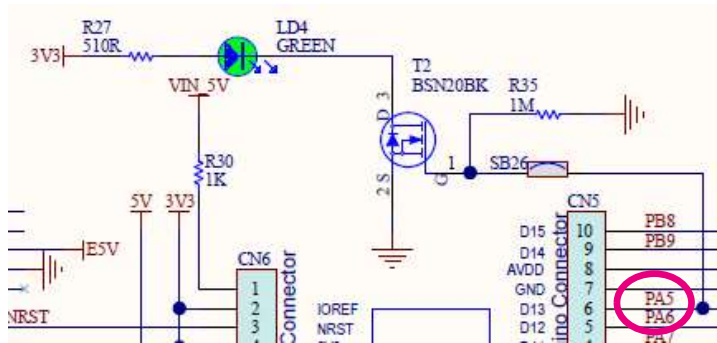


Give a name to the project

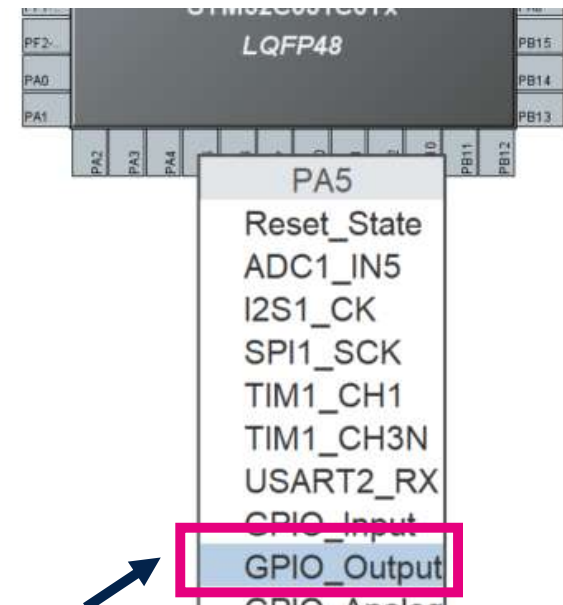


Pin configuration

- In this example we are going to use one of the LEDs present on the STM32C031 Nucleo board (connected to PA5 as seen in the schematic below)
- Search for PA5 in the search window at the bottom right 
- Left-click PA5 and set it to GPIO_Output mode

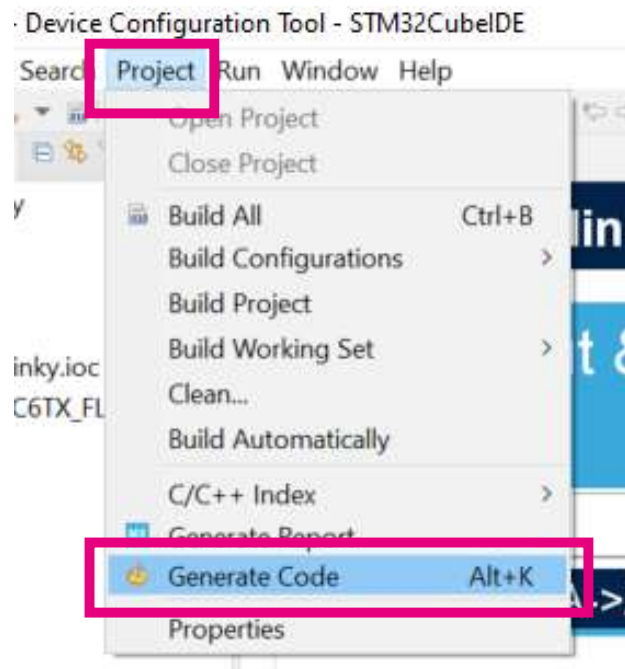


From Nucleo schematics



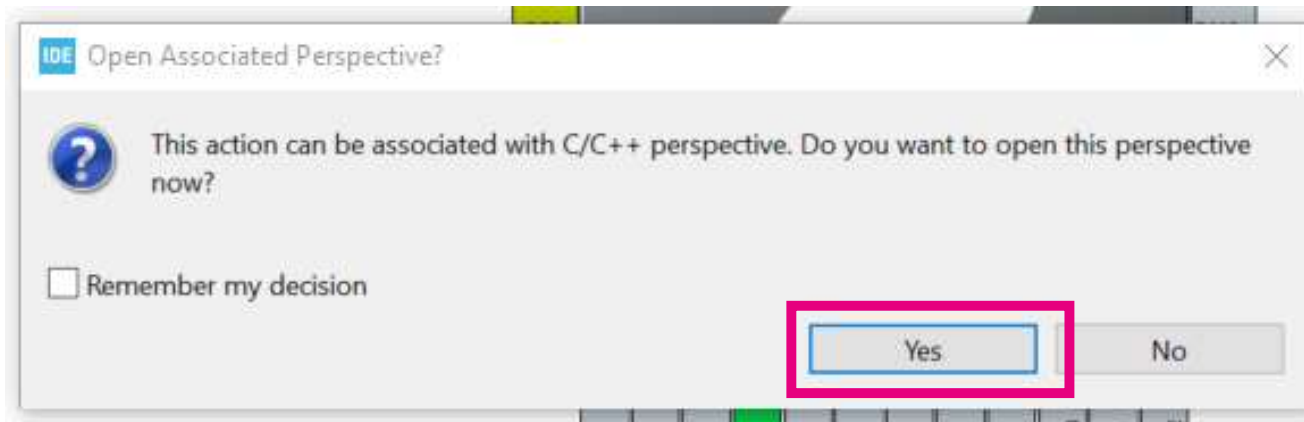
Generate source code

- Project -> Generate Code or ALT + K or Save the Project



Change Perspective

- Click Yes

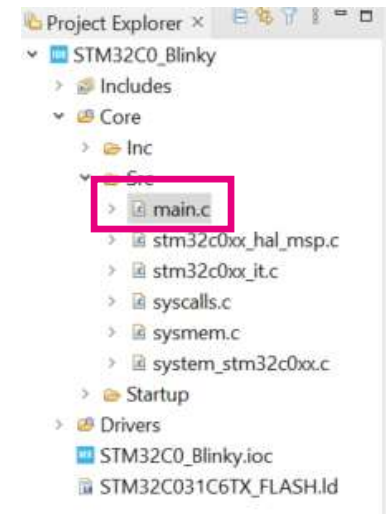


Step 4: toggle the LED

- In the Project Explorer Window:
- Expand the file tree and open the main.c file
- Add the following code inside the while(1) loop in “main.c” between the “USER CODE BEGIN WHILE” and “USER CODE END WHILE”

```
HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);  
HAL_Delay(100);
```

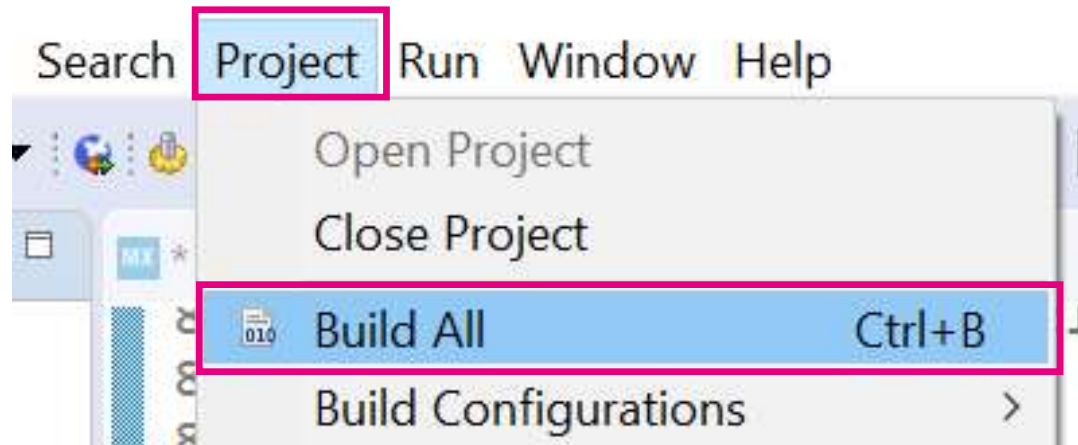
```
/* USER CODE BEGIN WHILE */  
while (1)  
{  
    HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);  
    HAL_Delay(100);  
}  
/* USER CODE END WHILE */
```



Note: Code within the “USER CODE BEGIN WHILE” / “USER CODE END WHILE” section will be preserved after regeneration by STM32CubeIDE

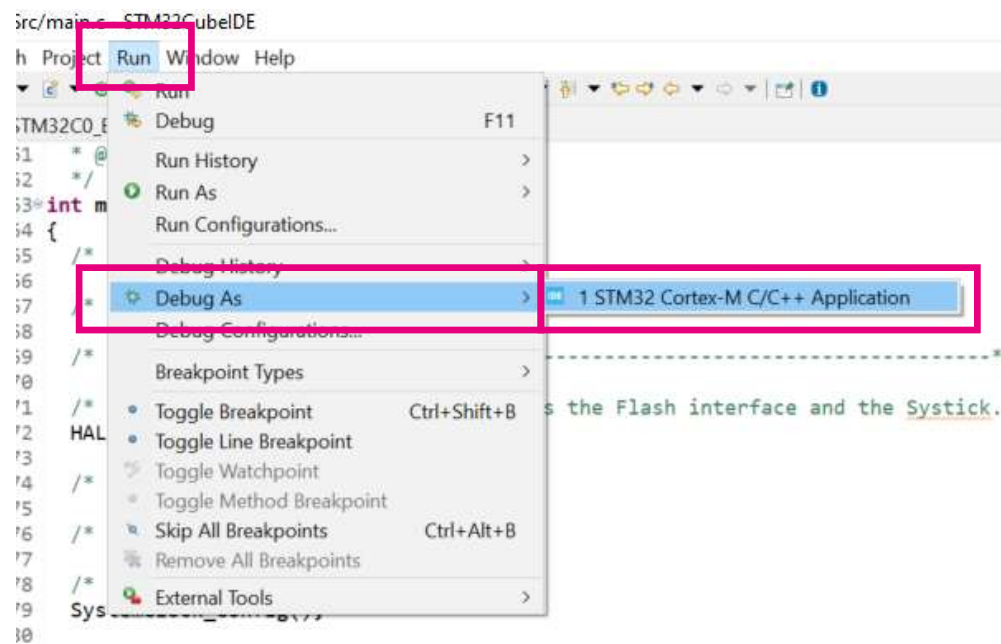
Build the project

- Project -> Build All or CTR +B



Debug

- Run -> Debug As -> STM32 Cortex-M C/C++ Application



ST-LINK Firmware Update (if needed)

ST-LINK firmware verification

This version of STM32CubeIDE provides a newer firmware version of the attached ST-LINK. Proceed with update?

☒ Always check for latest version.

Yes No

STLinkUpgrade 3.3.7

ST-LINK/V2-1 Refresh device list

Open in update mode

ST-Link ID: 0672FF505448826687073

Current Firmware: Unknown

Type: Unknown

Version: Unknown

Update to Firmware: Unknown

Upgrade

STLinkUpgrade 3.3.7

ST-LINK/V2-1 Refresh device list

Open in update mode

ST-Link ID: 0672FF5054488266870738

Current Firmware: Unknown

Type: STM32 Debug+Mass storage+VCP

Version: V2J38M27

Update to Firmware: V2J38M27 STM32

Upgrade

STLinkUpgrade 3.3.7

ST-LINK/V2-1 Refresh device list

Open in update mode

ST-Link ID: 0672FF505448826687073847

Current Firmware: Unknown

Type: STM32 Debug+Mass storage+VCP

Version: V2J39M27

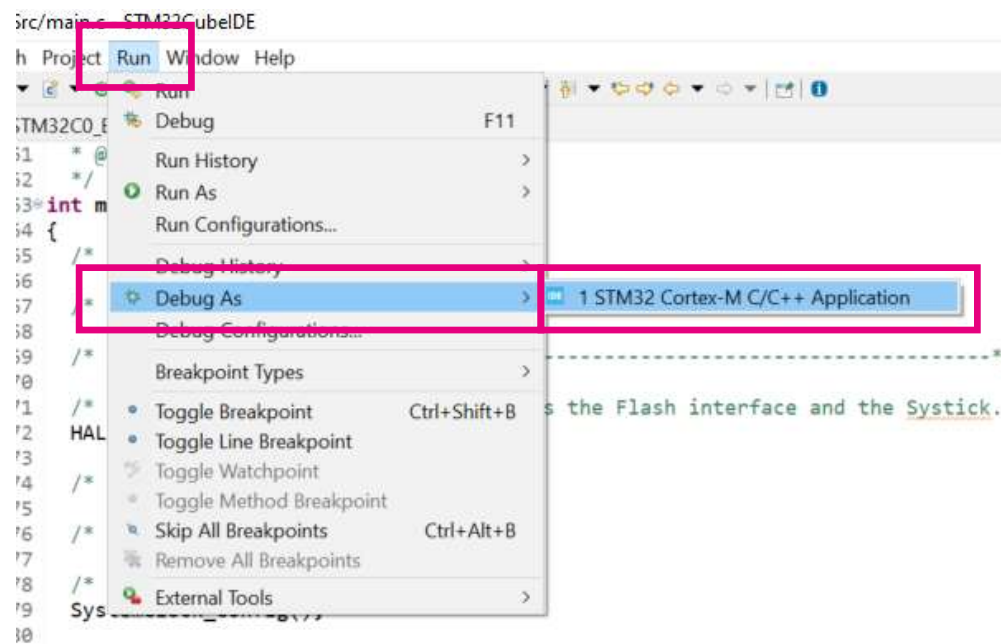
Update to Firmware: V2J39M27 STM32 Debug+Mass storage+VCP

Upgrade

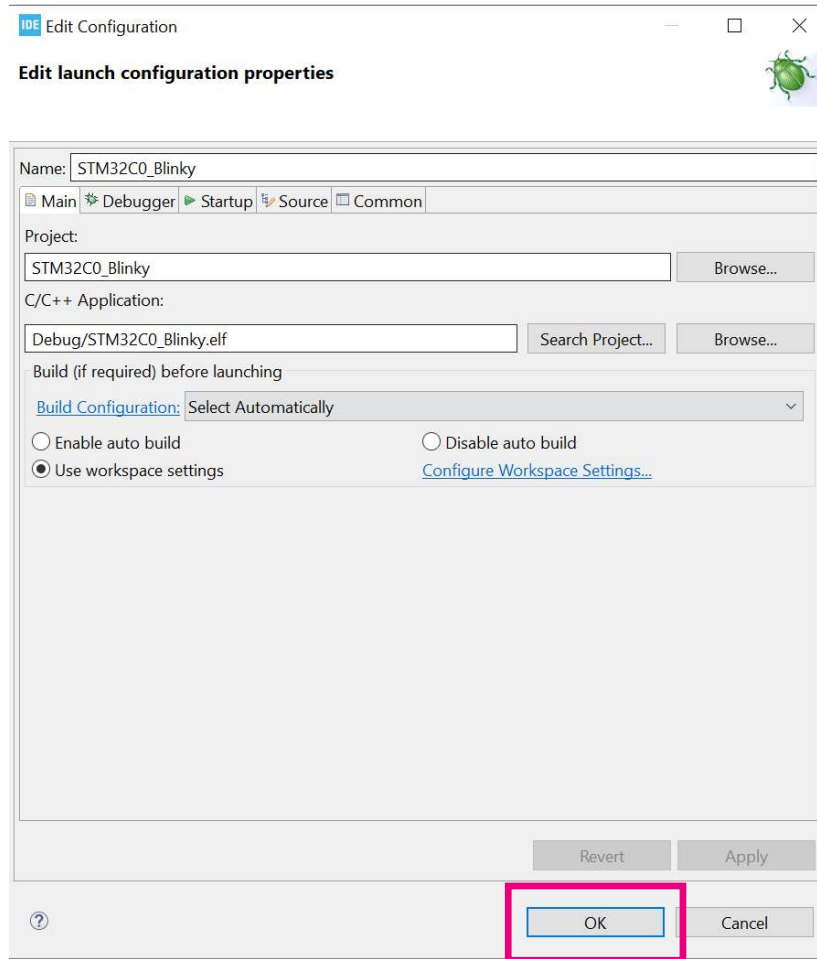
Upgrade successful.

Debug

- Run -> Debug As -> STM32 Cortex-M C/C++ Application



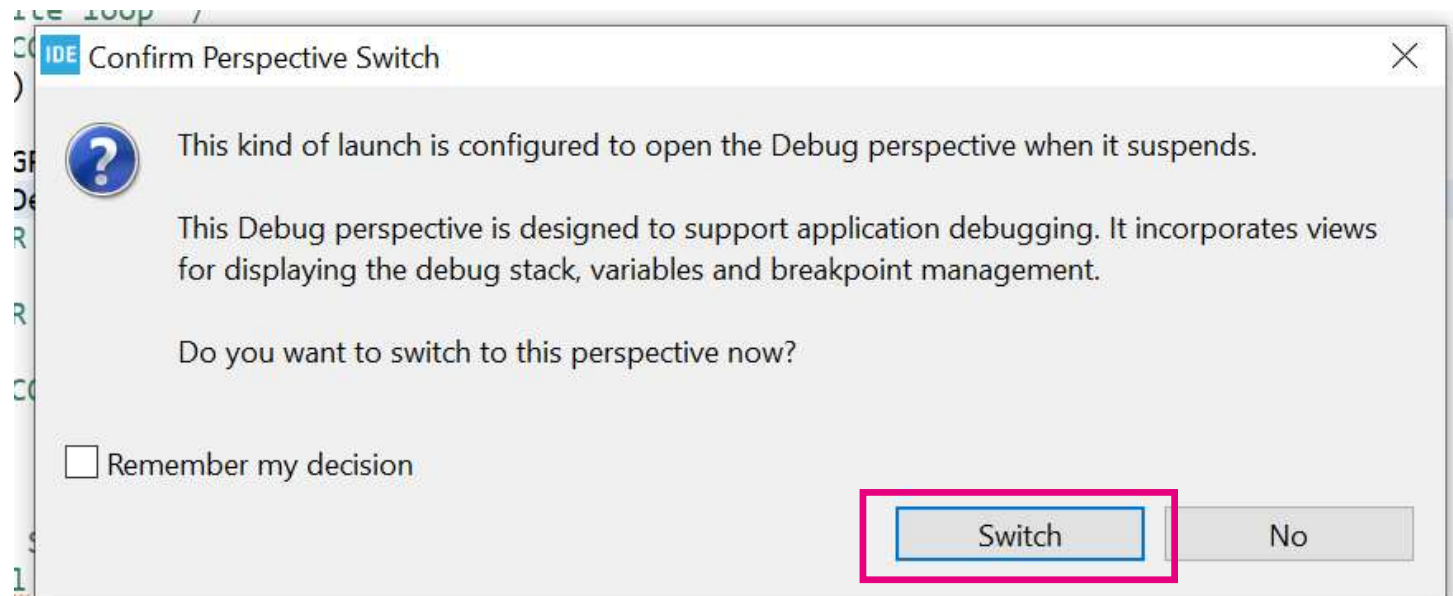
Check configuration



Note: It might ask you to upgrade the ST-LINK Driver once you press OK, do the upgrade as requested

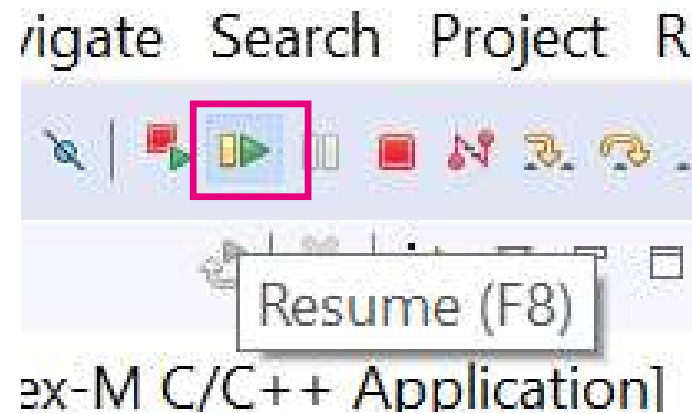
Switch the perspective

- Click Switch



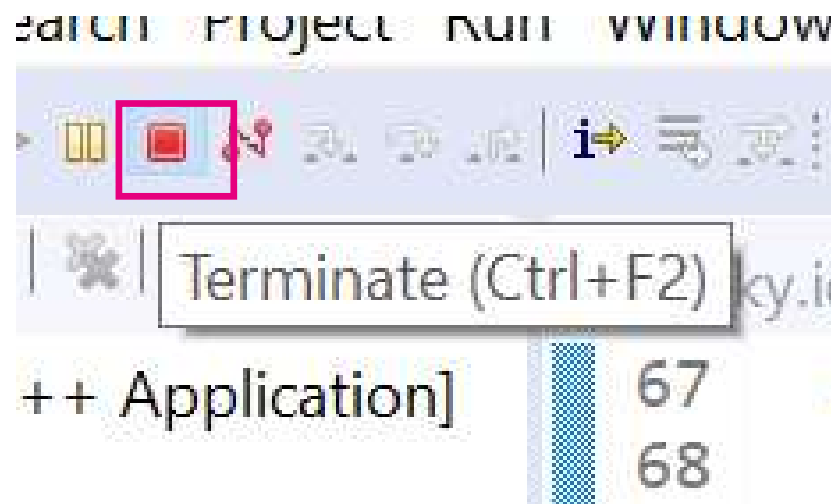
Run the code

- Click on the Resume Icon or F8

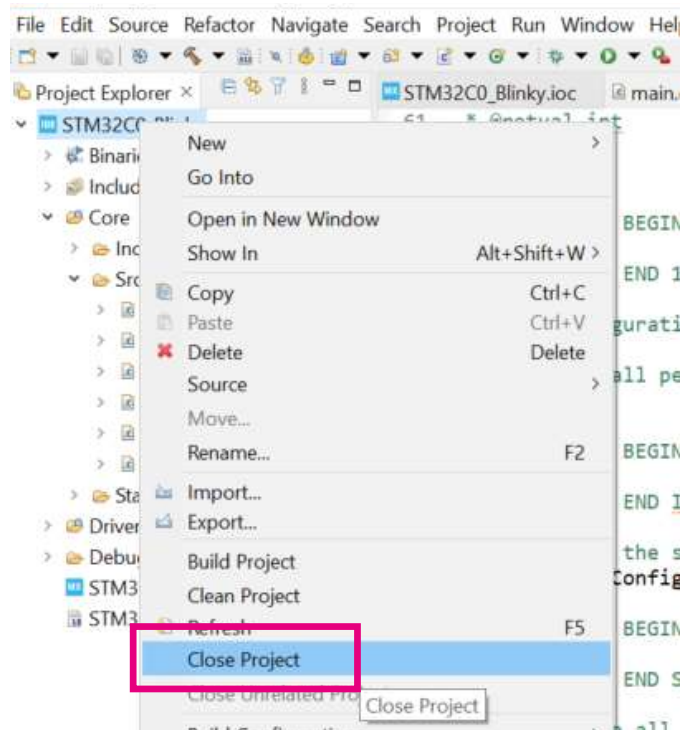


Enjoy the LED (LD4) blinking!

Exit the debugger



Close the project





life.augmented

STM32C0 MOOC Lab 2: PWM

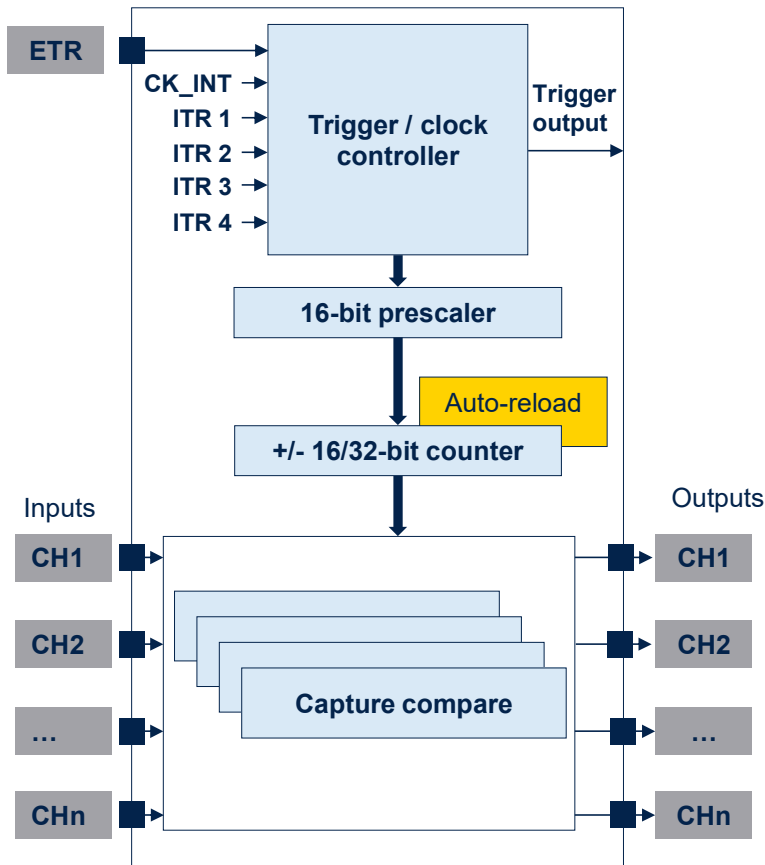
Nicolas FILLON

Lab: PWM timer

Objective:

- Now let's use a more advanced peripheral like the Timer.
- In this lab we are going to configure a Timer in a PWM (Pulse Width Modulation) mode to blink the LED that we previously controlled with a GPIO.
- PA5 has an alternate Timer channel alternate function which is Timer 1 Channel 1: TIM1_CH1 that we will be using.

Timer - overview



- Multiple timer units providing timing resources

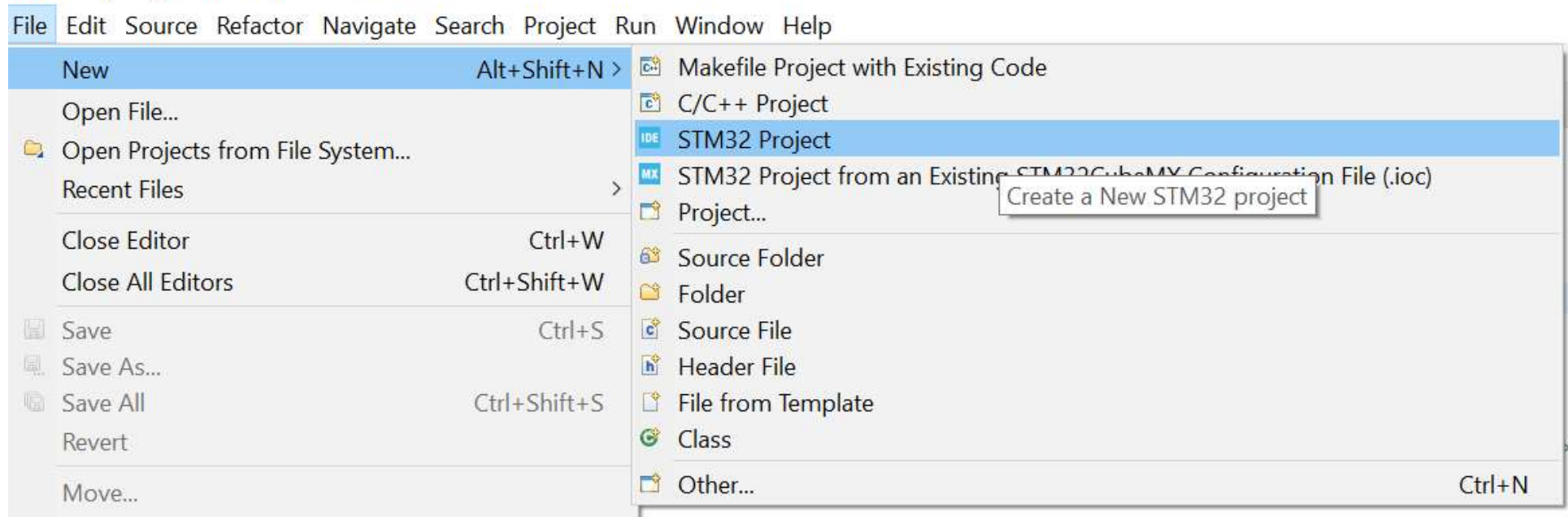
- Internally (triggers and time-bases)
- Externally, for outputs or inputs:
 - For waveform generation (PWM)
 - For signal monitoring or measurement (frequency or timing)

Application benefits

- Versatile operating modes reducing CPU burden and minimizing interfacing circuitry needs
- A single architecture for all timer instances offers scalability and ease-of-use
- Also fully featured for motor control and digital power conversion applications

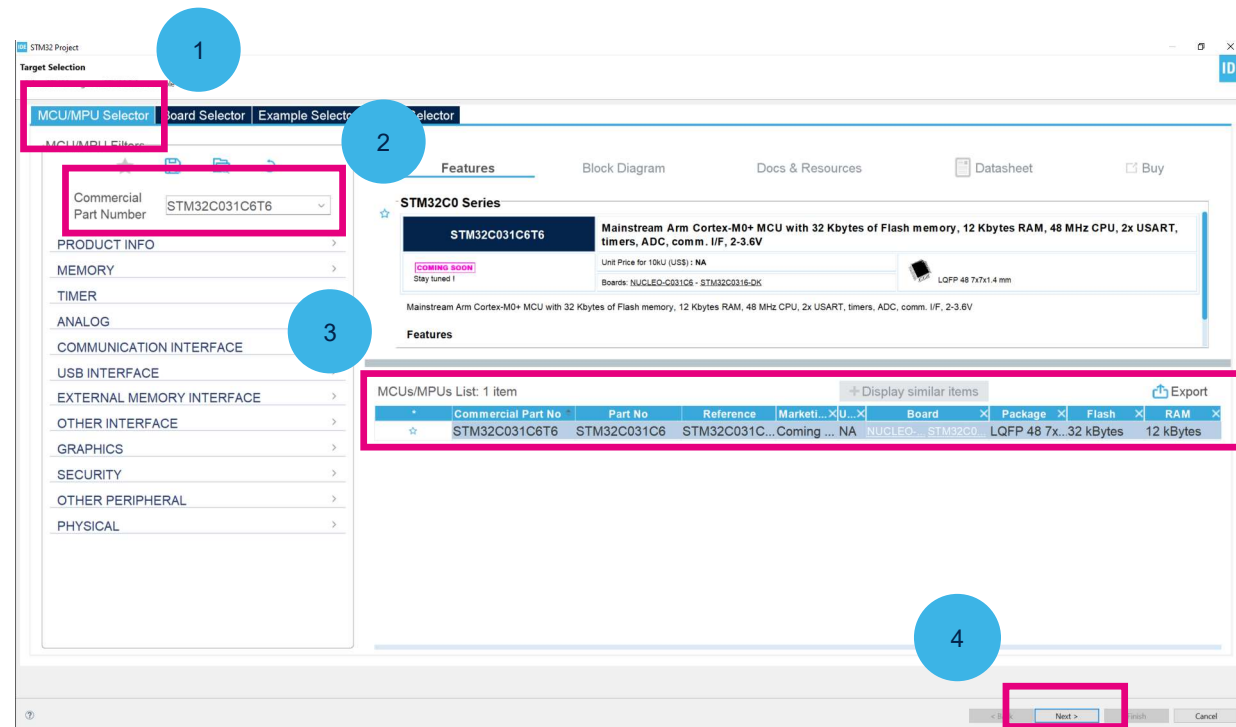
Lab: new project

- In STM32CubeIDE Create a New Project

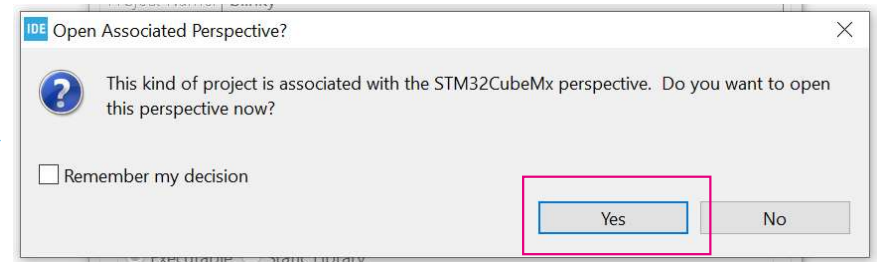
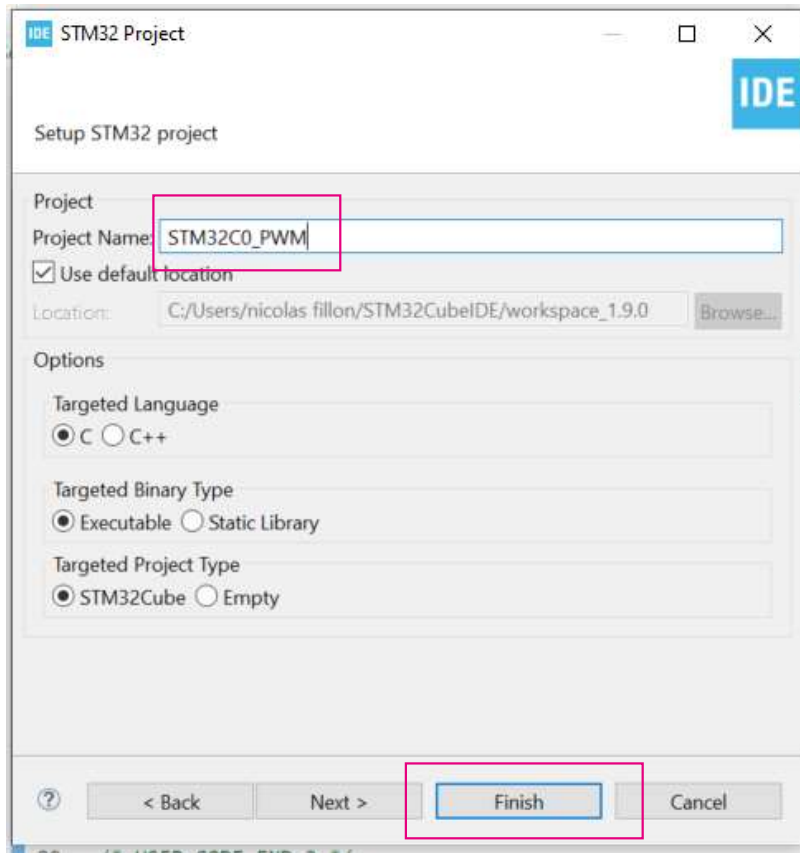


Create new project

- Click Access To MCU Selector 1
- Type: “STM32C031C6T6” in the Part Number Search 2
- Then Select STM32C031C6T6 3
 - LQFP48, 32 kB Flash 3
- Click “Next” 4

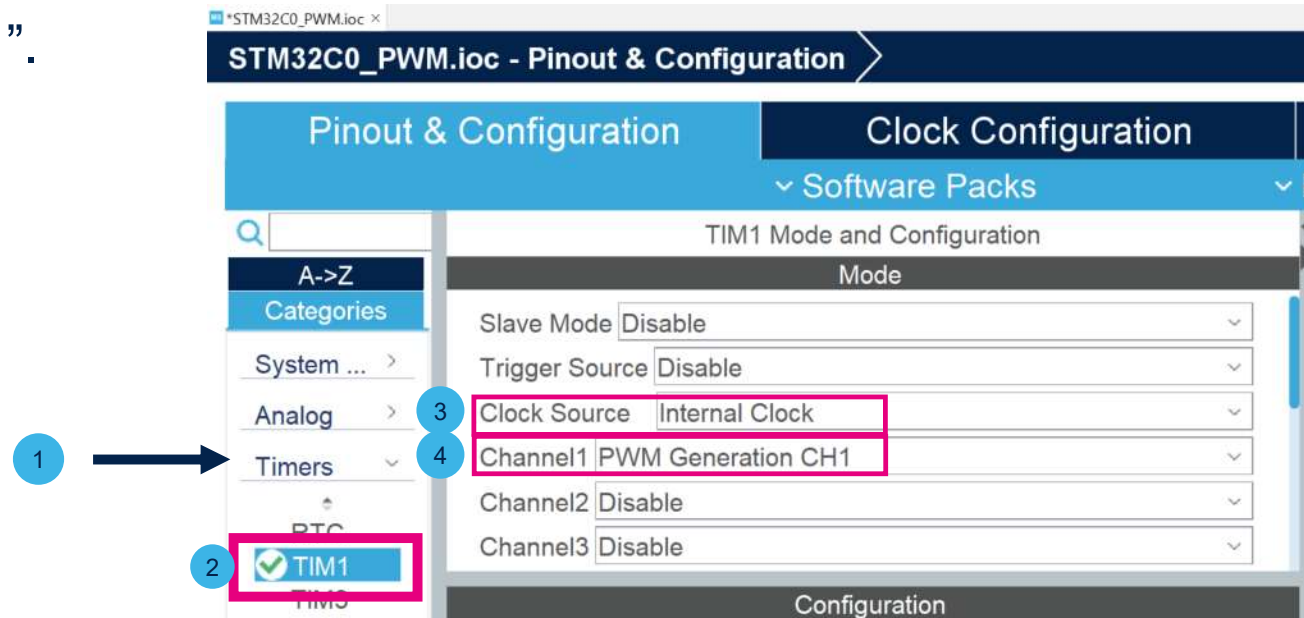


Give a name to the project



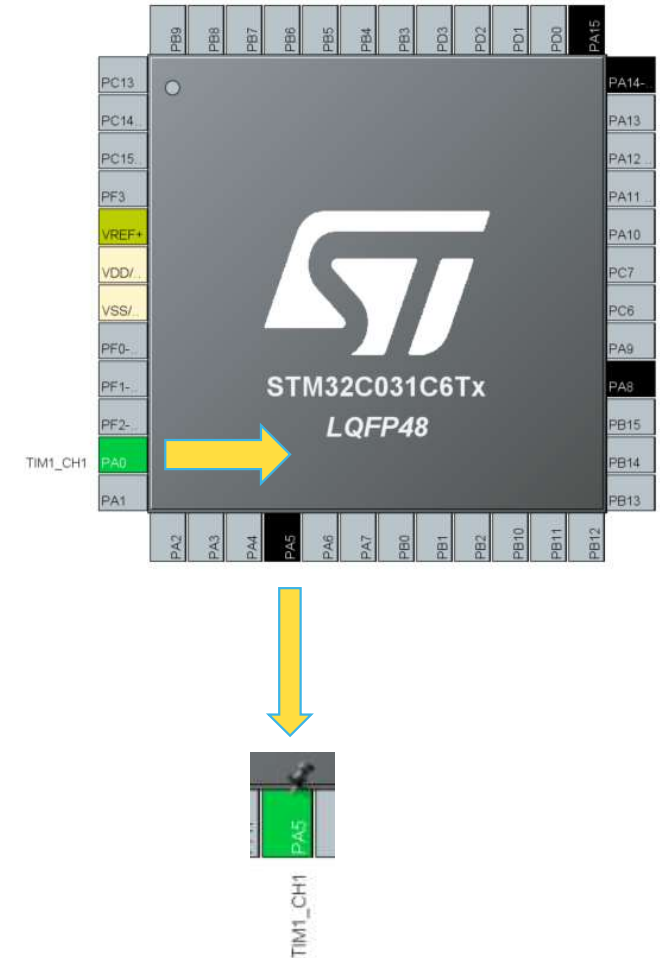
Lab: Timer 1 CH1 configuration

- In this STM32CubeIDE project we are going to add Timer 1 Channel 1 to blink LD4 (PA5) on the Nucleo board.
- In the Pinout & Configuration tab, Expand Timers Categories, then click on TIM1 peripheral and select “Internal Clock” for Source Clock and set Channel1 to “PWM Generation CH1”.



Remapping Timer 1 CH1 output to PA5

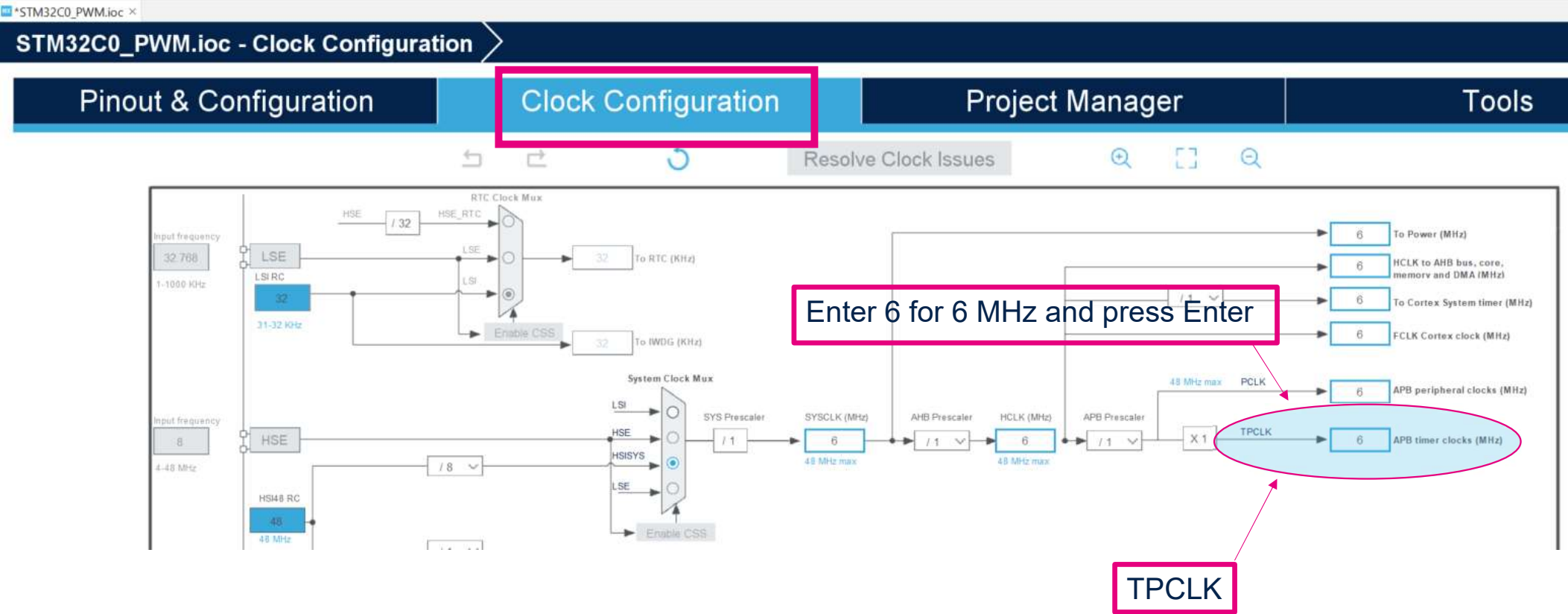
- By default, the tool will configure Timer 1 CH1 to PA0
- We want to remap it to PA5
 - NOTE: PA5 is connected to LD4 (Green LED)
- Hold “Ctrl” button and left mouse click on PA0
- Then drag the mouse pointer to PA5 and then release



Timer parameters calculation

- We want the Timer's PWM output channel to be:
 - $T = 1$ second period (1 Hz)
 - $D = 50\%$ duty cycle (0.5)
- Timer input clock frequency (TPCLK) is set to 6 MHz.
- Prescaler for the Timer is set to 128. The resulting timer counter clock is:
 - $CK_CNT = TPCLK / \text{Prescaler} = 6 \text{ MHz} / 128 = 46875 \text{ Hz}$
- To get $T=1$ Hz (or 1 sec period) the Counter Period needs to be set to: 46875
 - $\text{Counter Period} = CK_CNT / T = 46875 / 1 = 46875$
- To get $D=50\%$ duty cycle the Pulse needs to be set to: 23437
 - $\text{Pulse} = \text{Counter Period} / 2 = 46875 / 2 = 23437$

Clock tree configuration



TIM1 configuration – 4 steps

- Select the Pinout & Configuration
- In Parameters Settings of the TIM1 1
- Configure 1 Hz timer
 - PSC Prescaler -1 = 127 2
 - Counter Period = 46875 3
- Set CH1 PWM
 - Pulse = 23437 4

Note about Prescaler: $prescaler = PSC + 1$

Configuration

Reset Configuration

Parameter Settings 1

Configure the below parameters :

Search (Ctrl+F)

Prescaler (PSC - 16 bits value)	127	2
Counter Mode	Up	
Counter Period (AutoReload Register - 16 bits value)	46875	3
Internal Clock Division (CKD)	No Division	
Repetition Counter (RCR - 16 bits value)	0	
auto-reload preload	Disable	
> Trigger Output (TRGO) Parameters		
> Break And Dead Time management - BRK Configuration		
> Break And Dead Time management - BRK2 Configuration		
> Break And Dead Time management - Output Configuration		
> Clear Input		
v PWM Generation Channel 1		
Mode	PWM mode 1	
Pulse (16 bits value)	23437	4
Output compare preload	Enable	
Fast Mode	Disable	
CH Polarity	High	

Generate source code and add code

- Generate Code by saving the project (File -> Save)
- Open the main.c, Add the following code before the while(1) loop in order to start the PWM Timer:

Note : within “USER CODE BEGIN 2” / “USER CODE END 2” section

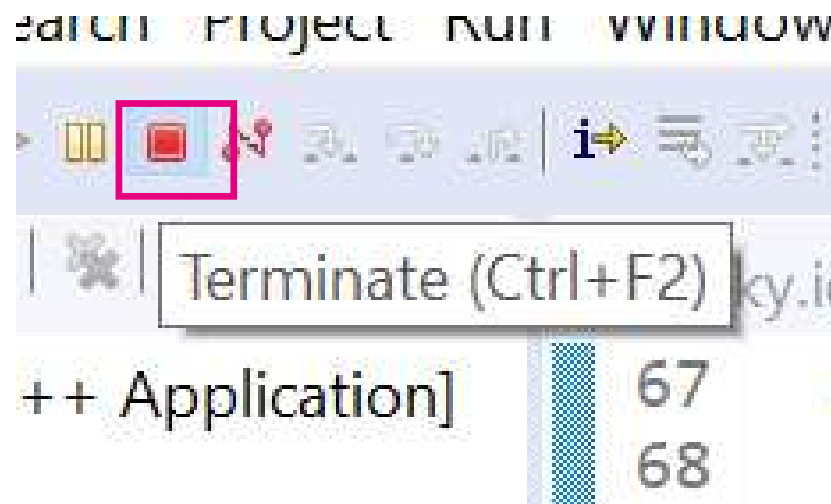
```
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
```

```
/* USER CODE BEGIN 2 */  
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);  
/* USER CODE END 2 */
```

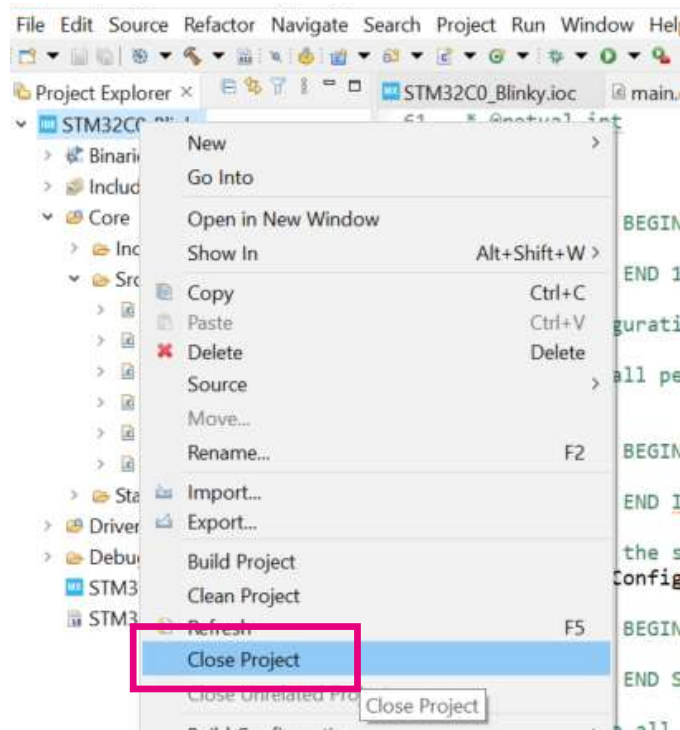
Build and run the code

- Build
- Enter debug session
- Run the code
- Enjoy the flashing LED (LD4)!
 - LD4 is flashing using the PWM Timer

Exit the debugger



Close the project





life.augmented

STM32C0 MOOC Lab 3: EXTI

Nicolas FILLON

Lab: NVIC + EXTERNAL Interrupts

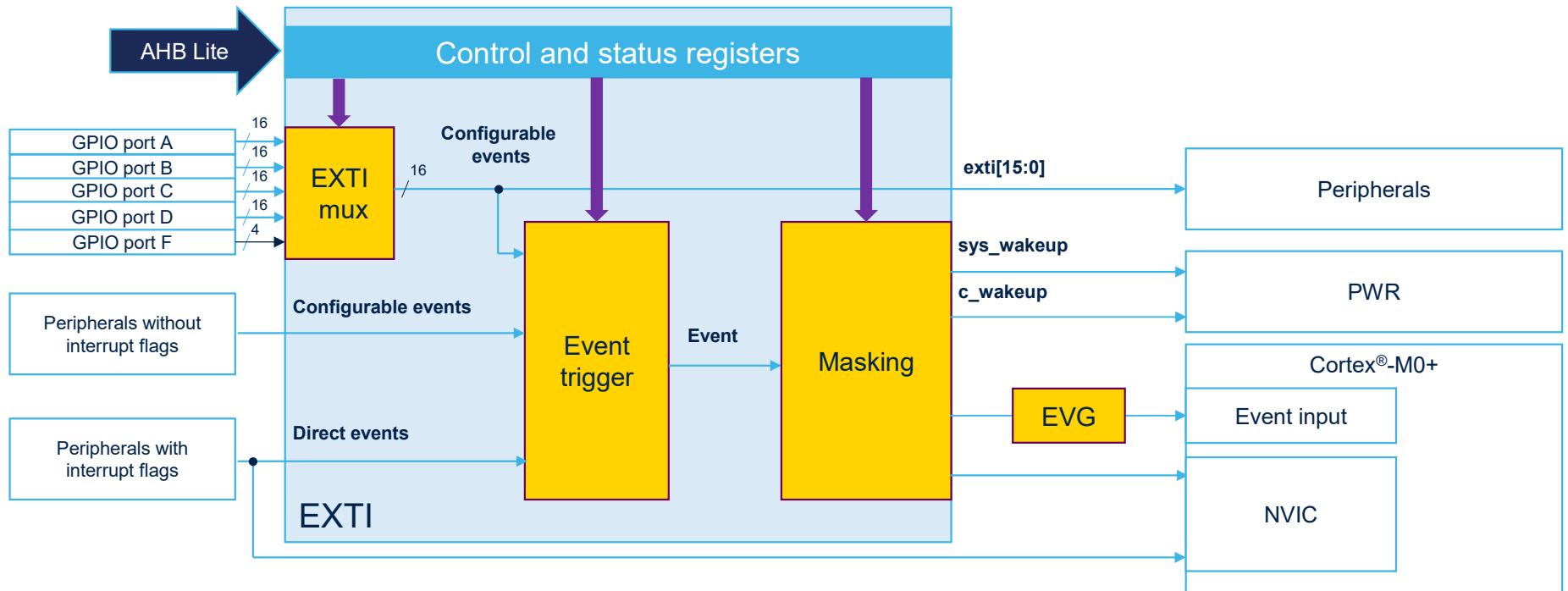
Objective:

- In this project we are going to configure the GPIO that is connected to the user button as External Interrupt (EXTI) with rising edge trigger.
- We will also configure the Interrupt Controller: the NVIC.

EXTI - key features

- Wake-up from Stop mode, interrupts and events generation
 - Independent interrupt and event masks
- Configurable events
 - Active edge selection
 - Dedicated pending flag
 - Trigger-able by software
 - Linked to:
 - GPIO, PVD, and COMPx
- Direct events
 - Status flag provided by related peripheral
 - Linked to:
 - RTC, TAMP, I2C1, USARTx, CEC, LPUART1, LPTIMx, LSE_CSS
- P[x]y (where x = {A,B,C ...} like as PA0, PB0, PC0... are linked to the same interrupt event EXTIy
- Groups of EXTI inputs with the same interrupt vector: (EXTI0_1, EXTI2_3, EXTI4_15)

EXTI - block diagram



EVG: EVent Generator

NVIC - overview

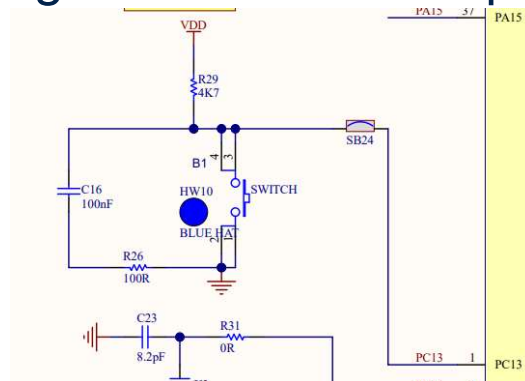
- The NVIC (Nested vector Interrupt Controller) is integrated in the Cortex®-M0+ CPU:
 - 32 maskable interrupt channels
 - 4 programmable priority levels
 - Low-latency exception and interrupt handling
 - Power management control

Application benefits

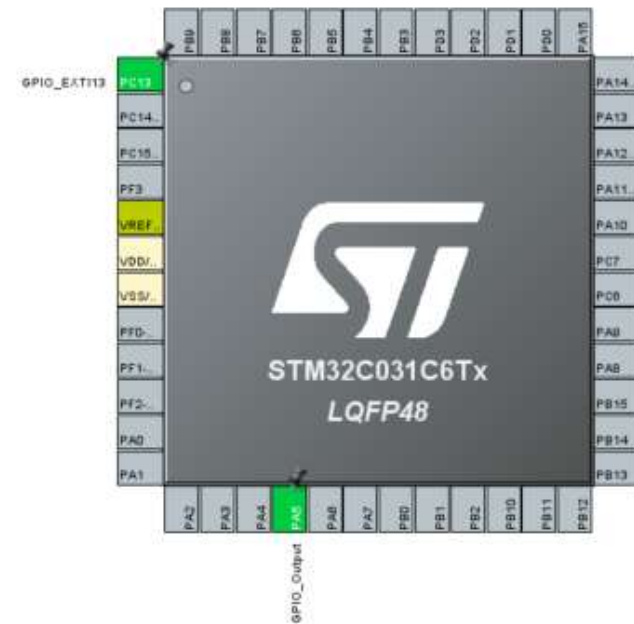
- Supports prioritization levels with dynamic control
- Fast response to interrupt requests
- Relocatable vector table

Lab: pinout configuration

- In STM32CubeIDE create a new project “STM32C0 EXTI”.
- Add configuration of the IO that is connected to the User Button (connected to PC13) and to toggle the LED LD4 (connected to PA5) on the STM32C0 Nucleo board.
- Left-click on PC13 and set it to GPIO_EXTI13 mode.
- PA5 to be configured as GPIO Output push-pull.

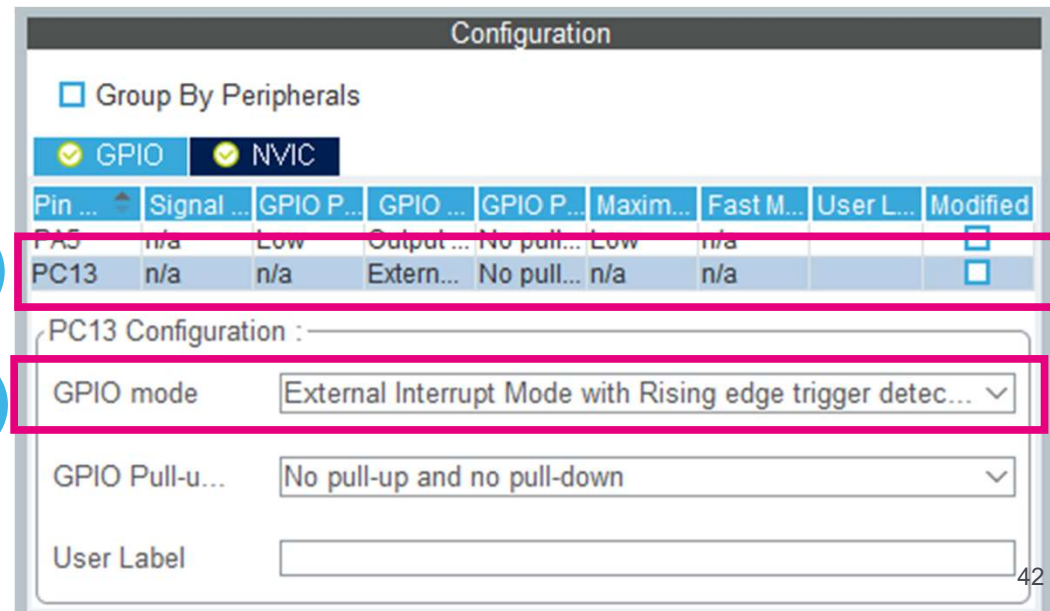
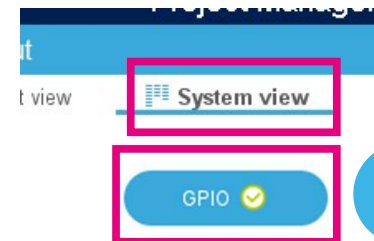


From Nucleo schematics



GPIO configuration

- Select GPIO under System View 1
- Click on Pin Name PC13 2
- Make sure GPIO mode is “External **Interrupt Mode** with Rising edge trigger detection” 3



NVIC configuration

- Select NVIC under System View



- Enable “EXTI line 4 to 15 interrupts” (by checking the box) 1

NVIC Mode and Configuration

Configuration

☒ NVIC ☒ Code generation

☐ Sort by Preemption Priority and Sub Priority

Search ☐ Show only enabled interrupts

NVIC Interrupt Table	Enabled	Preemption Priority
Non maskable interrupt	☒	0
Hard fault interrupt	☒	0
System service call via SWI instruction	☒	0
Pendable request for system service	☒	0
Time base: System tick timer	☒	0
PVD interrupt through EXTI line 16	☐	0
Flash global interrupt	☐	0
RCC global interrupt	☐	0
EXTI line 4 to 15 interrupts	☒	0

1

Generate source code and add code

- Generate Code
- Open main.c, add the following code:
 - within “USER CODE BEGIN PV” / “USER CODE END PV” section

```
uint8_t PC13_flag = 0;
```

```
/* USER CODE BEGIN PV */  
    uint8_t PC13_flag = 0;  
/* USER CODE END PV */
```

Add EXTI rising edge callback function

- Also, in main.c add the following code,
 - within “USER CODE BEGIN 4” / “USER CODE END 4” section

```
void HAL_GPIO_EXTI_Rising_Callback(uint16_t GPIO_Pin)
{
    PC13_flag++;
    if ( ( PC13_flag & 0x01 ) == 0x01 )
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
    }
    else
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
    }
}
```

```
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Rising_Callback(uint16_t GPIO_Pin)
{
    PC13_flag++;
    if ( ( PC13_flag & 0x01 ) == 0x01 )
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
    }
    else
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
    }
}
/* USER CODE END 4 */
```

Build and run the code

- Click the “Build” button
- Push the Blue “USER” button to toggle the LED LD4!



life.augmented

STM32C0 MOOC Lab 4: Printf debugging using UART

Nicolas FILLON

Lab: printf() debugging using UART

Objective:

- Redirect “printf “ output to USART2 which is connected to the ST-LINK Virtual COM port on the Nucleo board
- Using the Terminal in the STM32CubeIDE to view the printf output.

debugging settings

USART2 debug will be used via the ST-LINK Virtual-COM port

Set up additional GPIO / Clocks:

PA2 – USART2, “USART2-TX”

PA3 – USART2, “USART2-RX”

USART2 Clock = PCLK1 (48 MHz)

USART2 settings:

Asynchronous Mode

115200

8/N/1

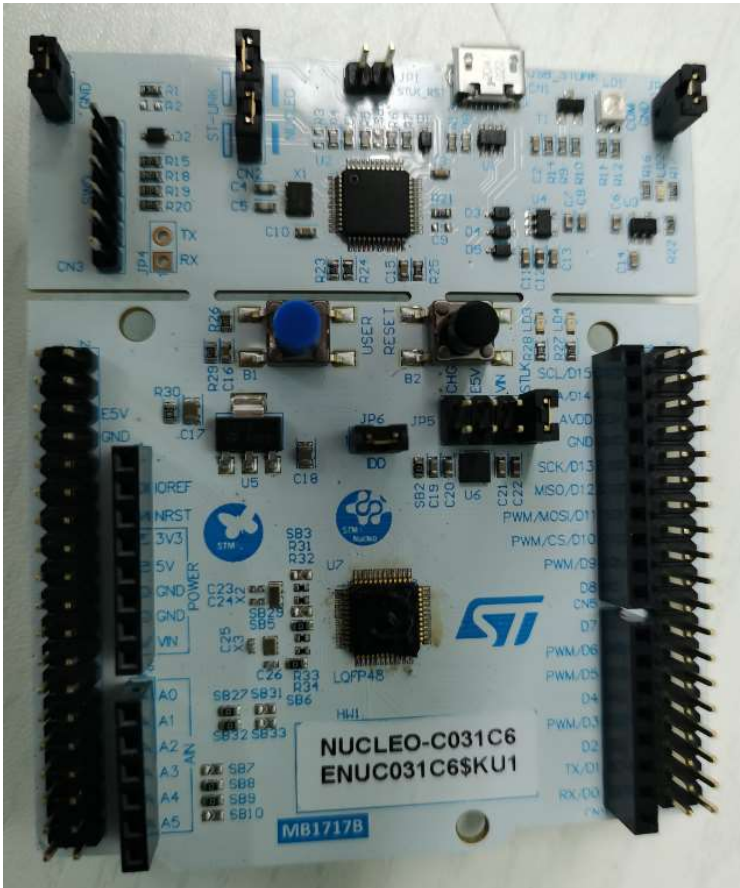
No HW Flow control

Tx/Rx,

No advanced features

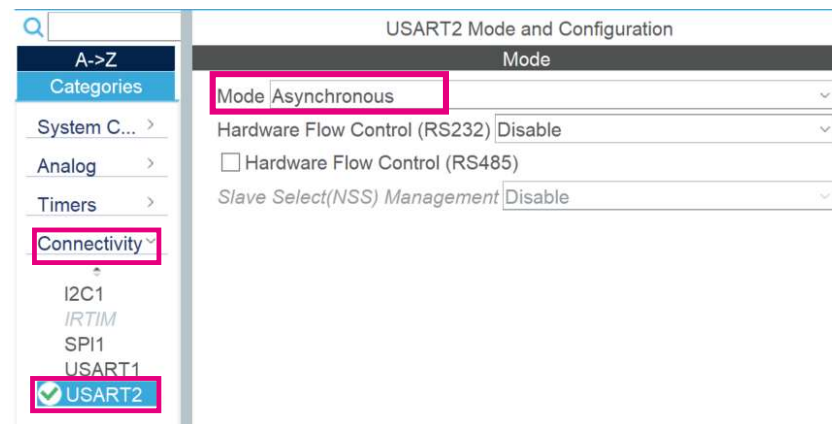
We will use the terminal in STM32CubeIDE.

USART2 is routed to the ST-LINK's USART, and brought via the USB Virtual-COM port class (SB27/32 located on the back on the board have been soldered)

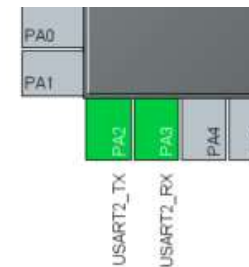


Create a new project and GPIO configuration additions

- Click on USART2 dialog (under Connectivity), and select Asynchronous mode:

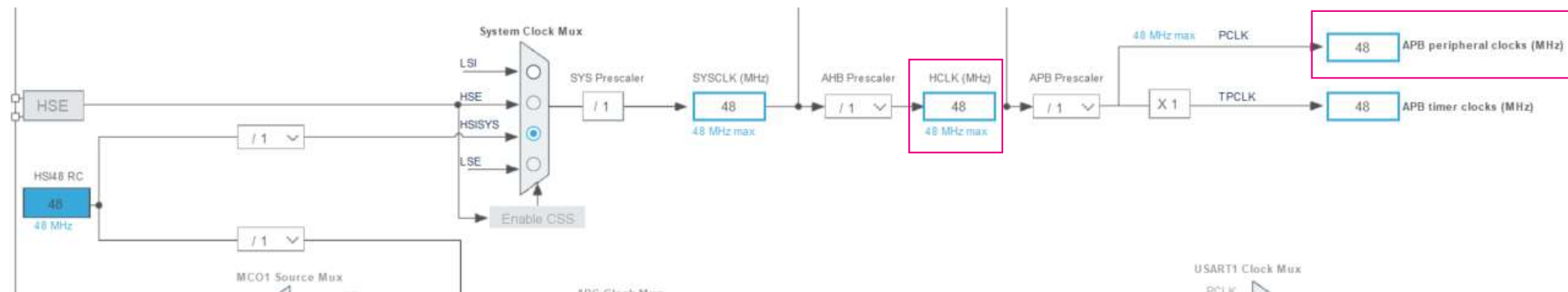


- Make sure PA2 & PA3 are selected for Tx / Rx pins:



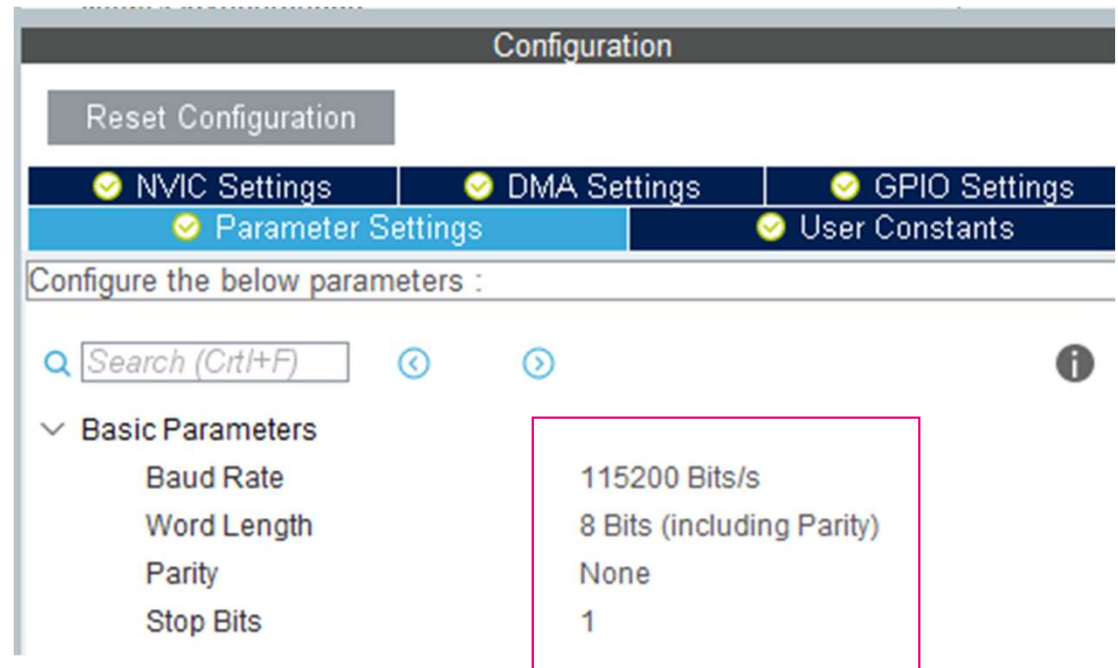
Clock configuration

- Run the STM32C0 at 48 MHz for this lab, the USART1 clock also at 48 MHz.



USART2 configuration

- Click on the Configuration tab and select USART2
- Parameter Settings tab
 - 115200 Bits/s
 - 8-bit word length
 - No parity bit
 - 1 Stop bit
 - Keep Default settings for the rest



Generate code and adding printf redirecting code in main.C

1- Add following code in the section below:

```
/* USER CODE BEGIN PFP */  
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)  
/* USER CODE END PFP */
```

2- Add following function in the section below:

```
/* USER CODE BEGIN 4 */  
PUTCHAR_PROTOTYPE  
{  
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, 0xFFFF);  
    return ch;  
}  
/* USER CODE END 4 */
```

```
/* USER CODE BEGIN PFP */  
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)  
/* USER CODE END PFP */
```

```
/* USER CODE BEGIN 4 */  
PUTCHAR_PROTOTYPE  
{  
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, 0xFFFF);  
    return ch;  
}  
/* USER CODE END 4 */
```

Adding application code in main.C

Add application code in main loop:

```
/* USER CODE BEGIN WHILE */  
while (1)  
{  
    printf("*** Hello World ** \n\r");  
    HAL_Delay(1000);  
/* USER CODE END WHILE */
```

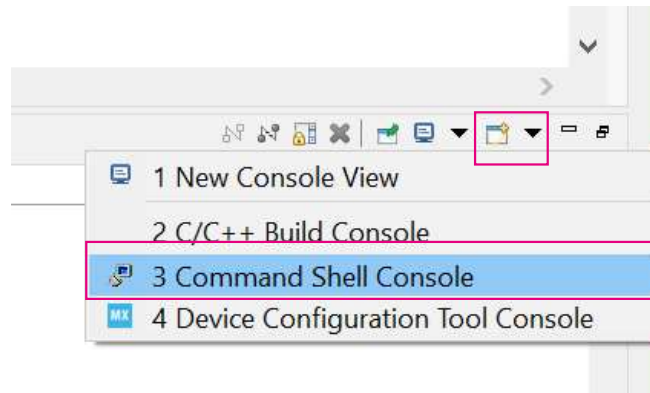
```
/* USER CODE BEGIN WHILE */  
while (1)  
{  
    printf("*** Hello World ** \n\r");  
    HAL_Delay(1000);  
/* USER CODE END WHILE */
```

Build the project and run the application

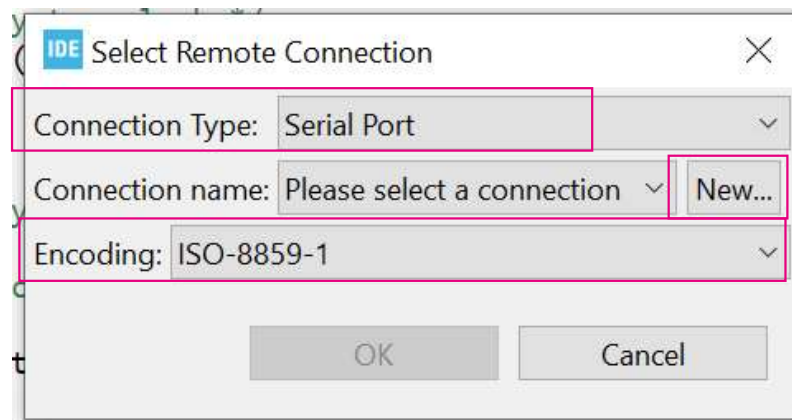
- Build Project
- Enter Debug session
- Click “Resume” button to run the code

Configure the Terminal inside STM32CubeIDE

- Open a Command Shell Console

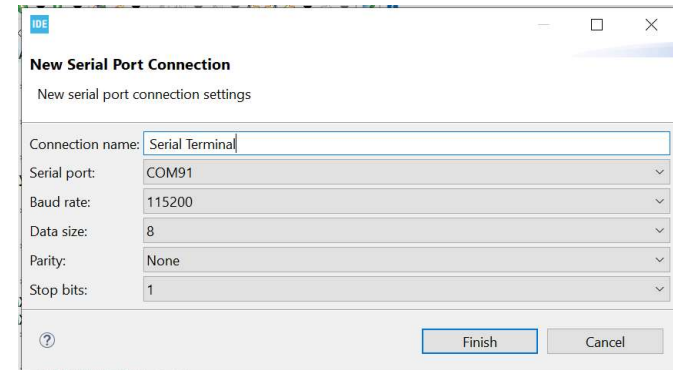


- In Select Remote Connection, select Serial Port for Connection Type, ISO for Encoding and then Press the “New...” icon below:

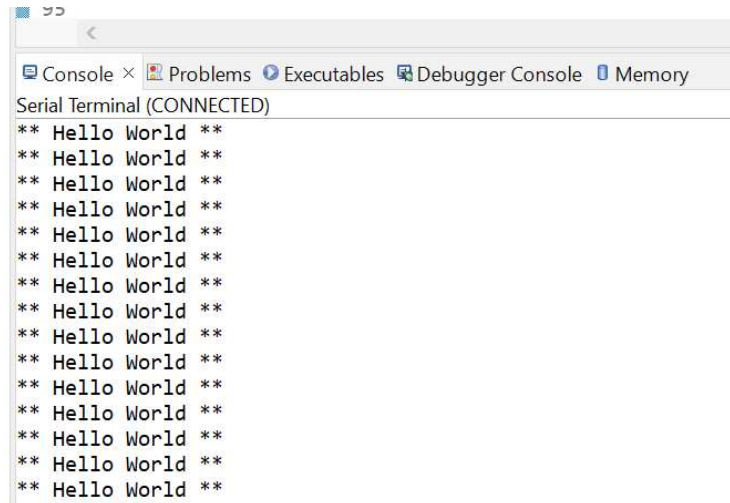


Configure the Terminal inside STM32CubeIDE

- In the New serial Port connection, give a name to the connection, select the COM port of your ST-LINK (COM91 for me) and use the following settings, then click Finish:



- You should see the printf message being displayed in the Console Tab.





life.augmented

STM32C0 MOOC Lab 5: LL Drivers

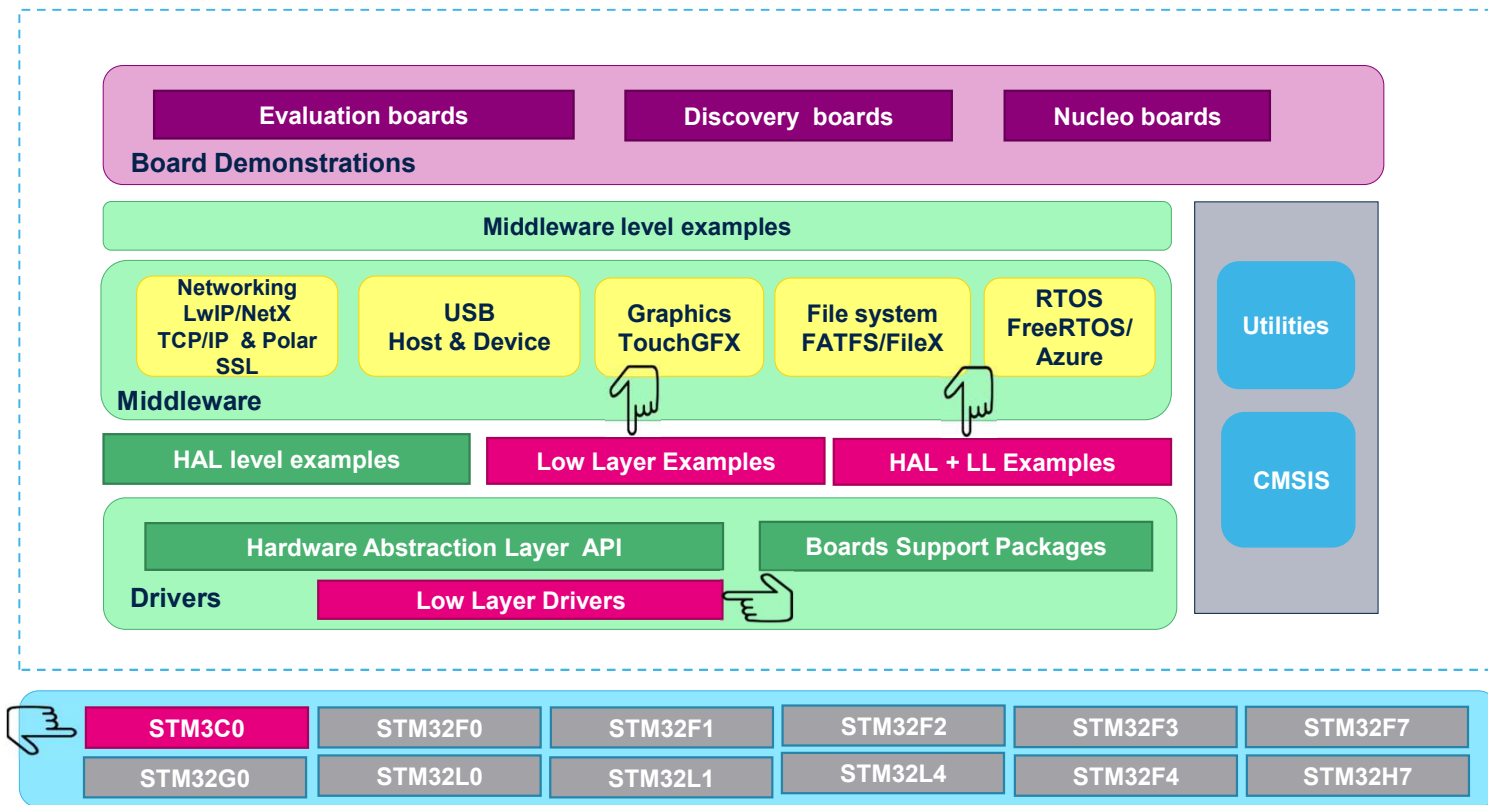
Nicolas FILLON

- STM32Cube HAL & LL are complementary and covers a wide range of applications requirements
- HAL offers high level and functionalities-oriented APIs, with high portability level and hide product/IPs complexity to end user
- LL offers low level APIs at registers level, w/ better optimization in code size and code execution but LL are less portability and require deep knowledge of the product specification

Low Layer (LL) library features

- The Low Layer (LL) Library offers the following services:
 - Set of static inline functions for direct register access (provided in *.h files only)
 - One-shot operations that can be used by the HAL drivers or from application level.
 - Independent from HAL and can be used standalone (without HAL drivers)
 - Full feature coverage of the supported peripherals

STM32Cube FW package block view



Benchmark- USART transmit example

- The below data are based on the “USART Transmitter IT” example:
 - Configure GPIO & USART peripheral for sending characters to HyperTerminal (PC) in Asynchronous mode using IT
- Using below configuration:
 - Platform: STM32L486xx
 - Compiler : IAR
 - Optimization : High Size
 - Heap Size = 512 Bytes
 - Stack Size = 512 Bytes

	HAL Drivers	Low layer Drivers
read-only code memory (Bytes)	7206	2154
read-only data memory (Bytes)	204	94
read write data memory (Bytes) (*)	1408	1093

ROM Size
divided by ~4

RAM Size
reduction

- LL** offer smaller footprint & high performance but less portability & require expertise
- HAL** offer high level API (hide complexity) & portability but higher footprint & less performance

(*) to add Heap and Stack size for total RAM



life.augmented

STM32C0 Lab: Using the low layer (LL) drivers

Nicolas FILLON

Lab: Using the low layer drivers

Objective:

- Generate a project with STM32CubeIDE using the Low Layer Drivers and check how much improvement we get compared to a HAL project in term of Flash and RAM usage

- In STM32CubeIDE create a project you can call “STM32C0_LL”
- Set GPIO PA5 as Output Push-Pull like we did for the “STM32C0_Blinky” lab

Configuration

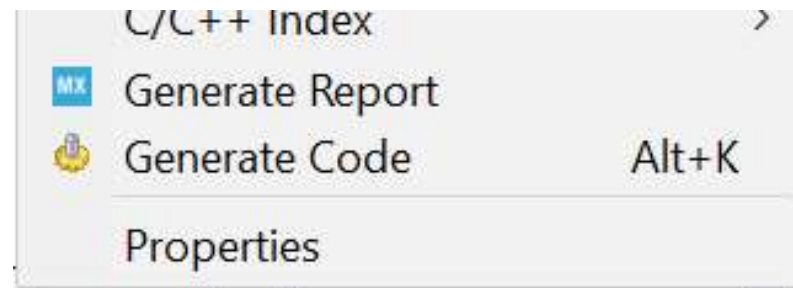
- In Project Manager Tab
- In the Advanced Settings Tab
- Instead of HAL drivers select LL (Low Level) for RCC, GPIO and CORTEX_M0+

The screenshot shows the STM32CubeIDE Project Manager tab. The 'Project Manager' tab is selected. The 'Driver Selector' section shows a search bar and a list of drivers: RCC, GPIO, and CORTEX_M0+. The 'LL' (Low Level) option is selected for each driver. The 'Generated Function Calls' table is also visible, showing the generated functions for the selected drivers.

Generate C...	Rank	Function Name	Peripheral Instanc...	Do Not Generate Function Ca	Visibility (Stat
☒	1	SystemClock_Co...	RCC	☐	☐
☒	2	MX_GPIO_Init	GPIO	☐	☒

Generate source code

- **Generate Code (ALT + K)**



Add code to toggle the LED

- Expand the “Drivers” and notice that now the drivers are low layer (_ll):

STM32C0xx_HAL_Driver

> Inc

▼ Src

- > stm32c0xx_ll_exti.c
- > stm32c0xx_ll_gpio.c
- > stm32c0xx_ll_pwr.c
- > stm32c0xx_ll_rcc.c
- > stm32c0xx_ll_utils.c

STM32C0_Blinky_LL

▼ Core

> Inc

▼ Src

> main.c

> stm32c0xx_hal_msp.c

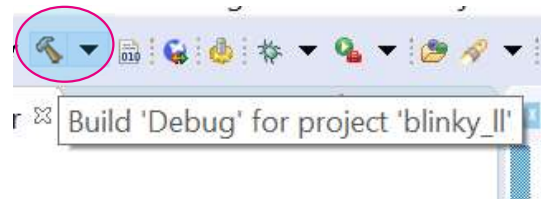
- Expand the file tree and open the main.c file
- Add the following code inside the while(1) loop
 - Add within “USER CODE BEGIN WHILE” / “USER CODE END WHILE” section (this will preserve your code after code regeneration)

```
LL_GPIO_TogglePin(GPIOA, LL_GPIO_PIN_5);  
// Delay 100 ms  
LL_mDelay(100);
```

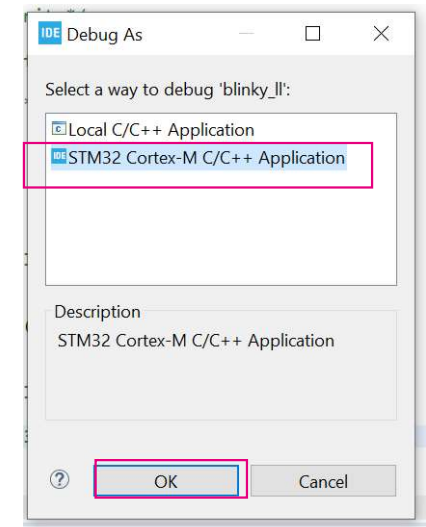
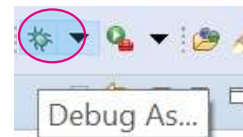
```
/* USER CODE BEGIN WHILE */  
while (1)  
{  
    LL_GPIO_TogglePin(GPIOA, LL_GPIO_PIN_5);  
    // Delay 100 ms  
    LL_mDelay(100);  
/* USER CODE END WHILE */
```

Build the project

- Click the “Build” button



- Click the “Start/Stop Debug Session” button



- Click “Run” button

- The Green LED (LD4) should be blinking just like the blinky example with the HAL drivers

Let's compare the differences between HAL and LL projects

- In the console window of the “STM32C0_Blinky” project that is using HAL drivers, after building we see:

CDT Build Console [STM32C0_Blinky]

20:36:24 **** Incremental Build of configuration Debug for project STM32C0_Blinky ****

make -j8 all

arm-none-eabi-size STM32C0_Blinky.elf

text	data	bss	dec	hex	filename
------	------	-----	-----	-----	----------

4440	20	1572	6032	1790	STM32C0_Blinky.elf
------	----	------	------	------	--------------------

Finished building: default.size.stdout

20:36:25 Build Finished. 0 errors, 0 warnings. (took 487ms)

Let's compare the map files between HAL and LL projects

- In the console window of the “STM32C0_Blinky_LL” project that is using LL drivers, after building we see:

```
CDT Build Console [STM32C0_Blinky_LL]
20:37:30 **** Incremental Build of configuration Debug for project STM32C0_Blinky_LL ****
make -j8 all
arm-none-eabi-size  STM32C0_Blinky_LL.elf
   text    data     bss      dec       hex filename
   3608      12    1564    5184    1440 STM32C0_Blinky_LL.elf
Finished building: default.size.stdout

20:37:30 Build Finished. 0 errors, 0 warnings. (took 424ms)
```

Comparison table between HAL and LL for our “blinky” code

- Using below configuration:
 - Heap Size = 512 Bytes / Stack Size = 512 Bytes
 - STM32Cube C0 1.0.0
 - CubeIDE 1.10.1

	HAL Drivers	Low layer Drivers	
read-only code memory (Bytes)	4440	3608	ROM Size with 20% improvment
read write data memory (Bytes)	20	12	RAM Size reduction divided ~ by 2



life.augmented

STM32C0 MOOC Lab 6: ADC + DMA + TIM

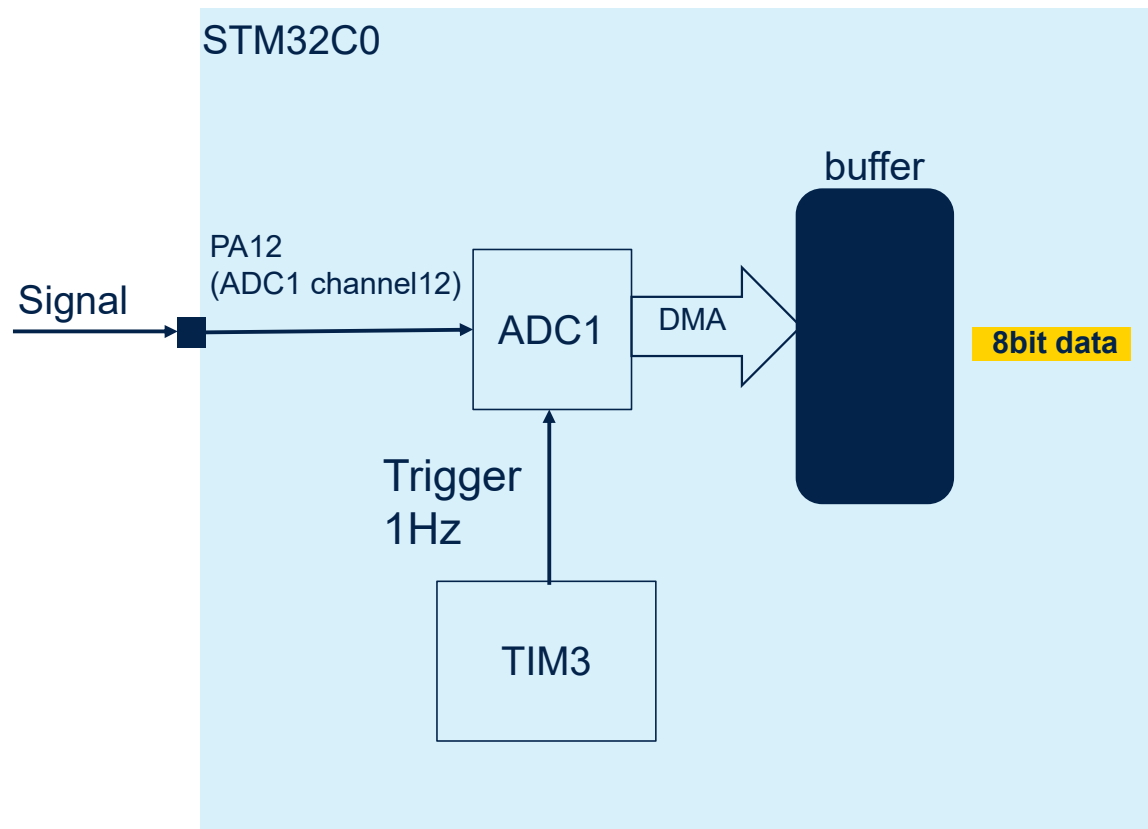
Nicolas FILLON

Lab: ADC+DMA+TIM

Objective:

- The objective of this lab is to learn about the ADC (Analog to Digital Converter) with the DMA (Direct Memory Access) and TIM (Timer) of the STM32C0.
- We will convert a signal on PA12 which is ADC1 Channel 12 and store the converted value in a buffer in RAM thanks to DMA and Timer.

Block Diagram



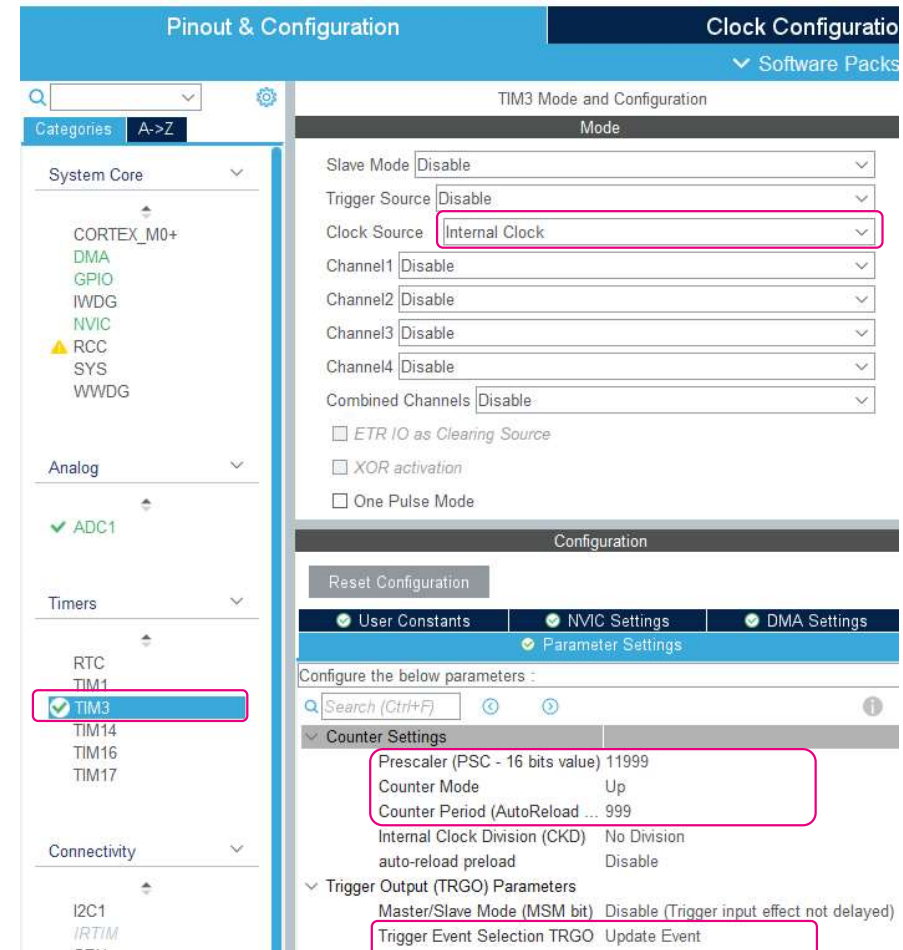
Example description

- ADC1 is measuring 8bit data on PA12 (ADC1_IN12 channel) each 1 second triggered by TIM3 (working as timebase) on its update event
- Data from ADC1 are transferred by DMA (DMA1_Channel2) to the buffer
- DMA1_Channel2 is configured in Circular mode using bytes (both sides)

TIM3 configuration

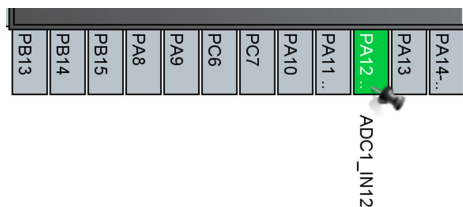
Goal: Configure TIM3 as upcounter using an internal clock which would overflow each 1 second with TRGO signal generation on this event.

- Select TIM3 within Timers group
- In Mode part:
 - Clock Source = Internal Clock
- In Configuration part:
 - Prescaler = 11999
 - Counter Period = 999
 - Trigger Event Selection TRGO = Update Event

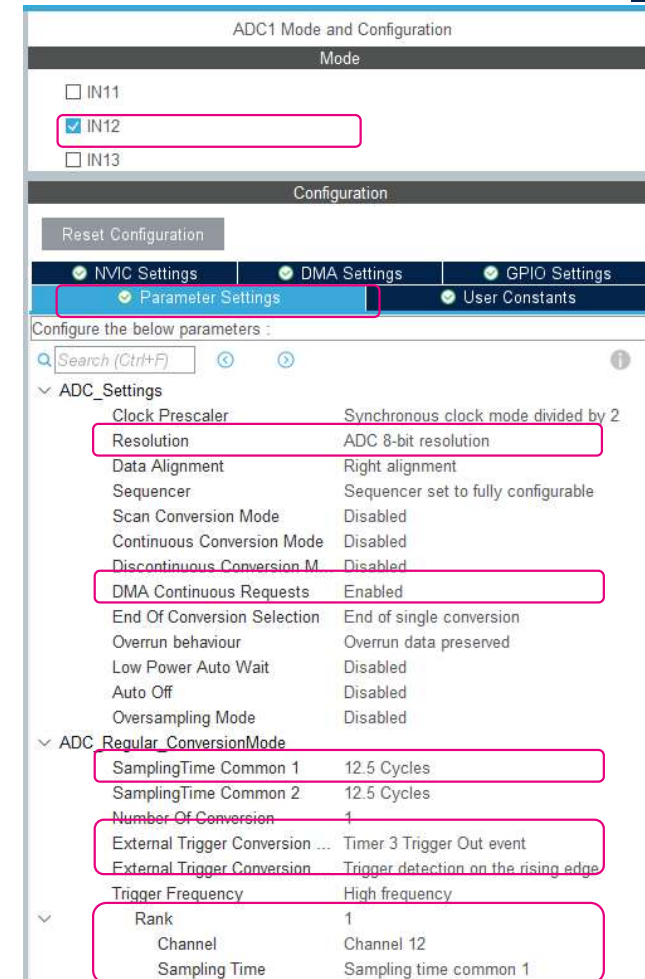


Goal: Configure ADC1 to convert signal on ADC1_IN12 line (PA12) on each trigger coming from TIM3 TRGO line, using 12.5 cycles sample time, 8bit resolution and transferring data over DMA.

- Select ADC1 within Analog group
- In Mode part select IN12 channel
- In Configuration part select Parameter Settings tab and change:
 - Resolution = 8 bits
 - DMA Continuous Requests = Enabled
 - SamplingTime Common1= 12.5 Cycles
 - Ext Trigger Conv Source = Timer 3 Trigger Out event (TRGO)
 - Ext Trigger Conversion Edge = rising edge
 - Rank1, Sampling time = Sampling time common 1



ADC1 configuration



ADC1 – DMA configuration

Goal: Add DMA support on ADC1 transfer complete events. Transfer using bytes using circular buffer

- Select DMA Settings tab within ADC Configuration
- Click on Add button and select ADC1 as DMA request
- Set the following DMA Settings:
 - Mode = Circular
 - Increment address = on Memory side
 - Data width = Byte on both sides

The screenshot shows the 'Configuration' window in STM32CubeMX. The 'DMA Settings' tab is selected. A table lists the DMA configuration:

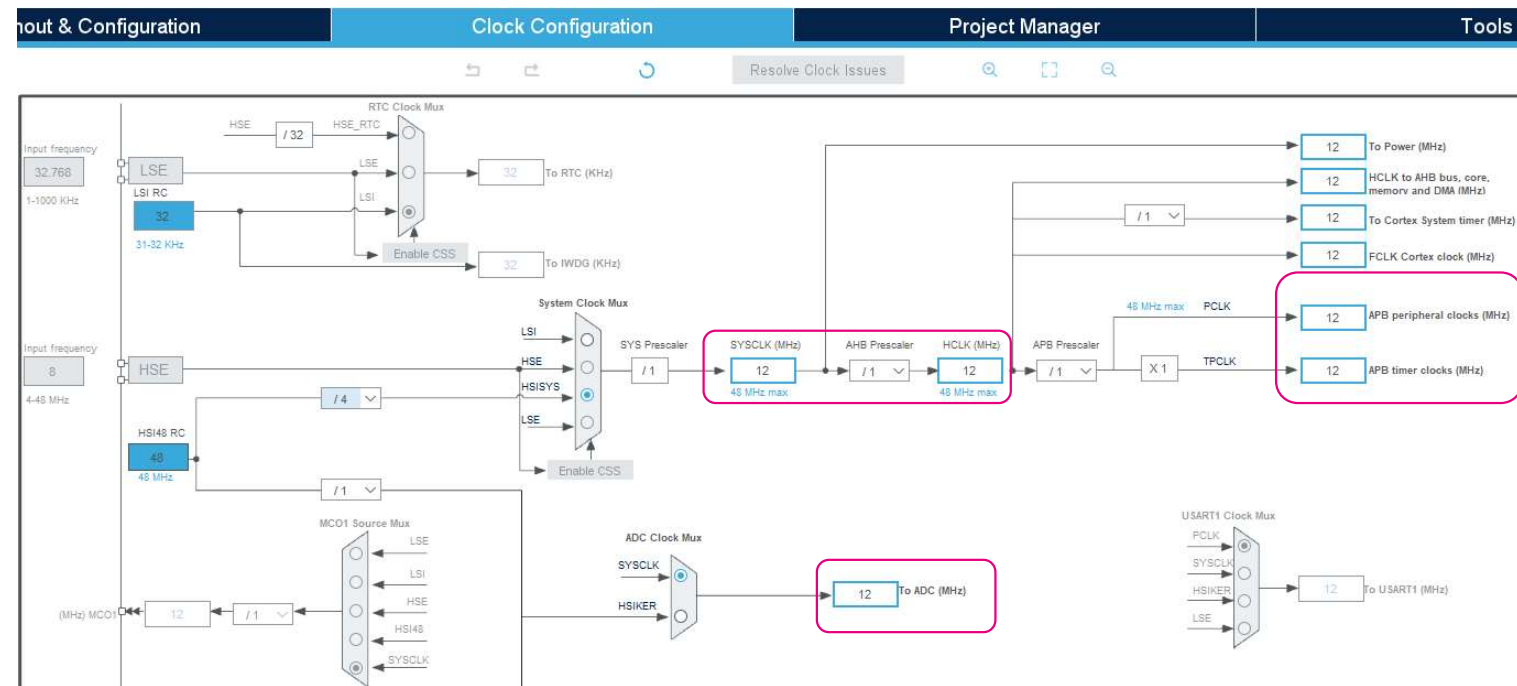
DMA Request	Channel	Direction	Priority
ADC1	DMA1 Channel 2	Peripheral To Memory	Low

Below the table, the 'Add' button is visible. The 'DMA Request Settings' section shows the following configuration:

- Mode: Circular (selected)
- Increment Address: ☐ (unchecked)
- Peripheral: ☐ (unchecked)
- Memory: ☒ (checked)
- Data Width: Byte (selected for both Peripheral and Memory)

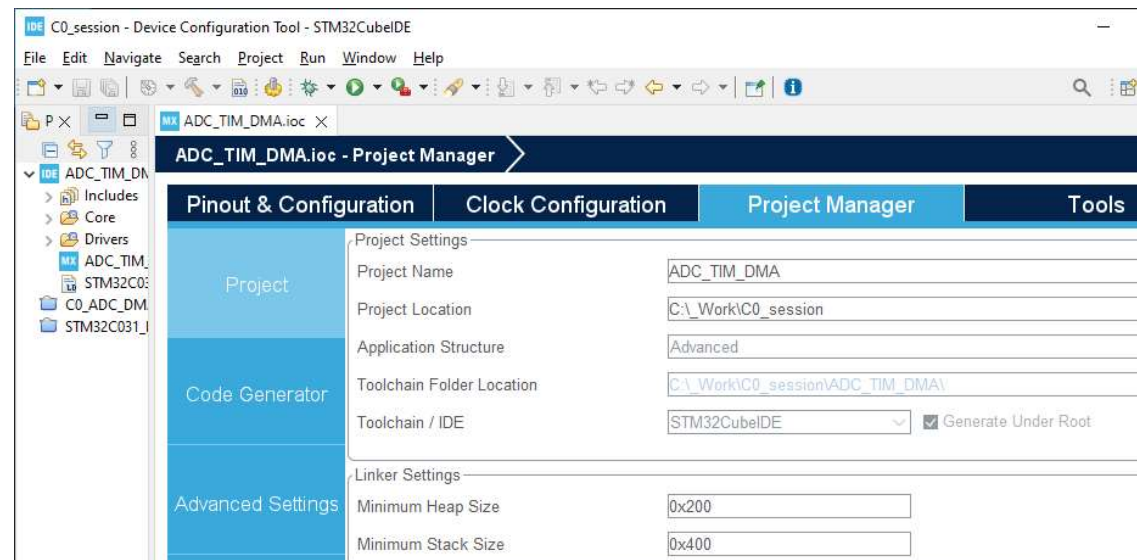
Clock Configuration default settings

- Default configuration on HSI 12MHz
- No need to change
- System clock 12MHz common for ADC1, TIM3



Project generation STM32CubeIDE

- Select „Project Manager” section
- Specify project location and project name if not yet done
- Generate the project by one of the following actions:
 - Pressing  icon
 - Saving project by „**Ctrl+S**”
 - Select **Project -> Generate project**
 - Click on **Alt+K**
- After project generation open main.c file



- Data buffer declaration

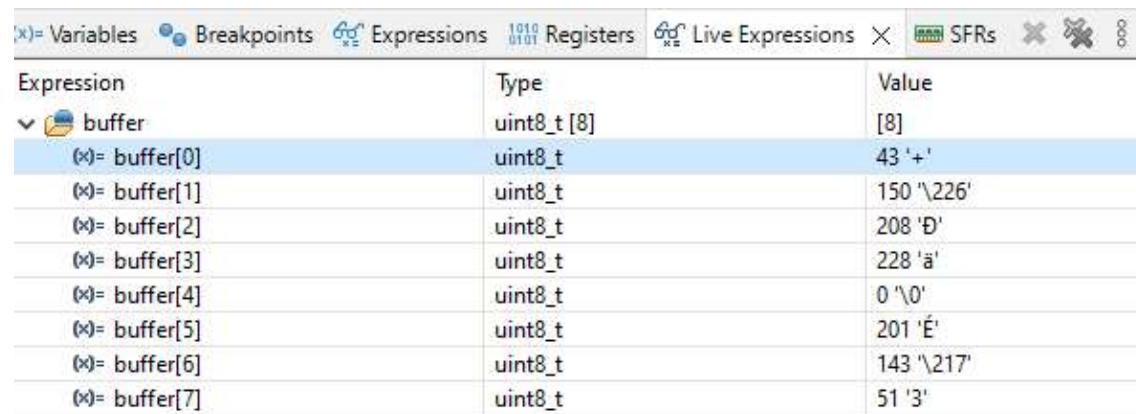
```
/* USER CODE BEGIN PV */  
uint8_t buffer[8];
```

- All peripherals start

```
/* USER CODE BEGIN 2 */  
HAL_ADCEx_Calibration_Start(&hadc1);  
HAL_ADC_Start_DMA(&hadc1, (uint32_t *)buffer, 8);  
HAL_TIM_Base_Start(&htim3);
```

Run the application

- Compile the code (i.e using „hammer” icon) 
- Start debug session (i.e using „bug” icon) 
- Add **buffer** table into „live expressions”
- Run the application
- As a result, you should see each second new data coming to **buffer** table within live expressions if you change PA12 input signal



Expression	Type	Value
buffer	uint8_t [8]	[8]
(x)= buffer[0]	uint8_t	43 '+'
(x)= buffer[1]	uint8_t	150 '\226'
(x)= buffer[2]	uint8_t	208 'Ð'
(x)= buffer[3]	uint8_t	228 'à'
(x)= buffer[4]	uint8_t	0 '\0'
(x)= buffer[5]	uint8_t	201 'É'
(x)= buffer[6]	uint8_t	143 '\217'
(x)= buffer[7]	uint8_t	51 '3'

In case it is not working, please have a look whether DMA is initialized BEFORE any other peripheral which is using it.



life.augmented

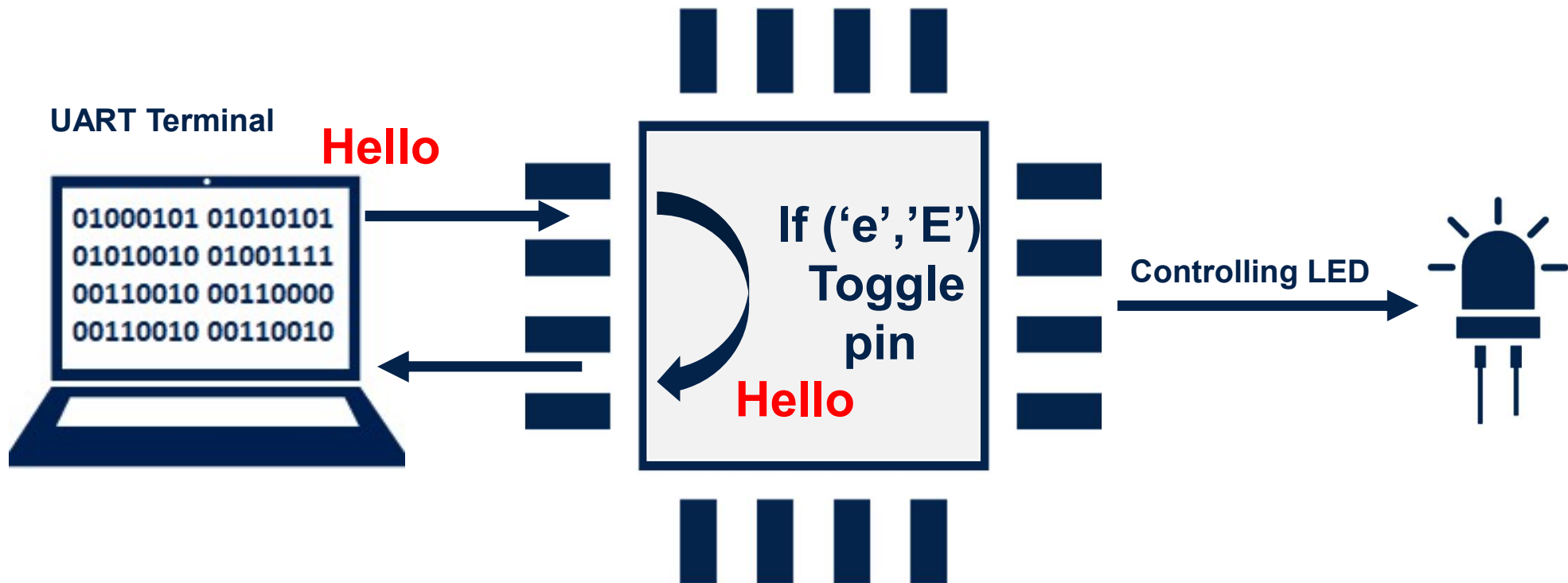
STM32C0 MOOC Lab 7: RAM

Nicolas FILLON

Objective:

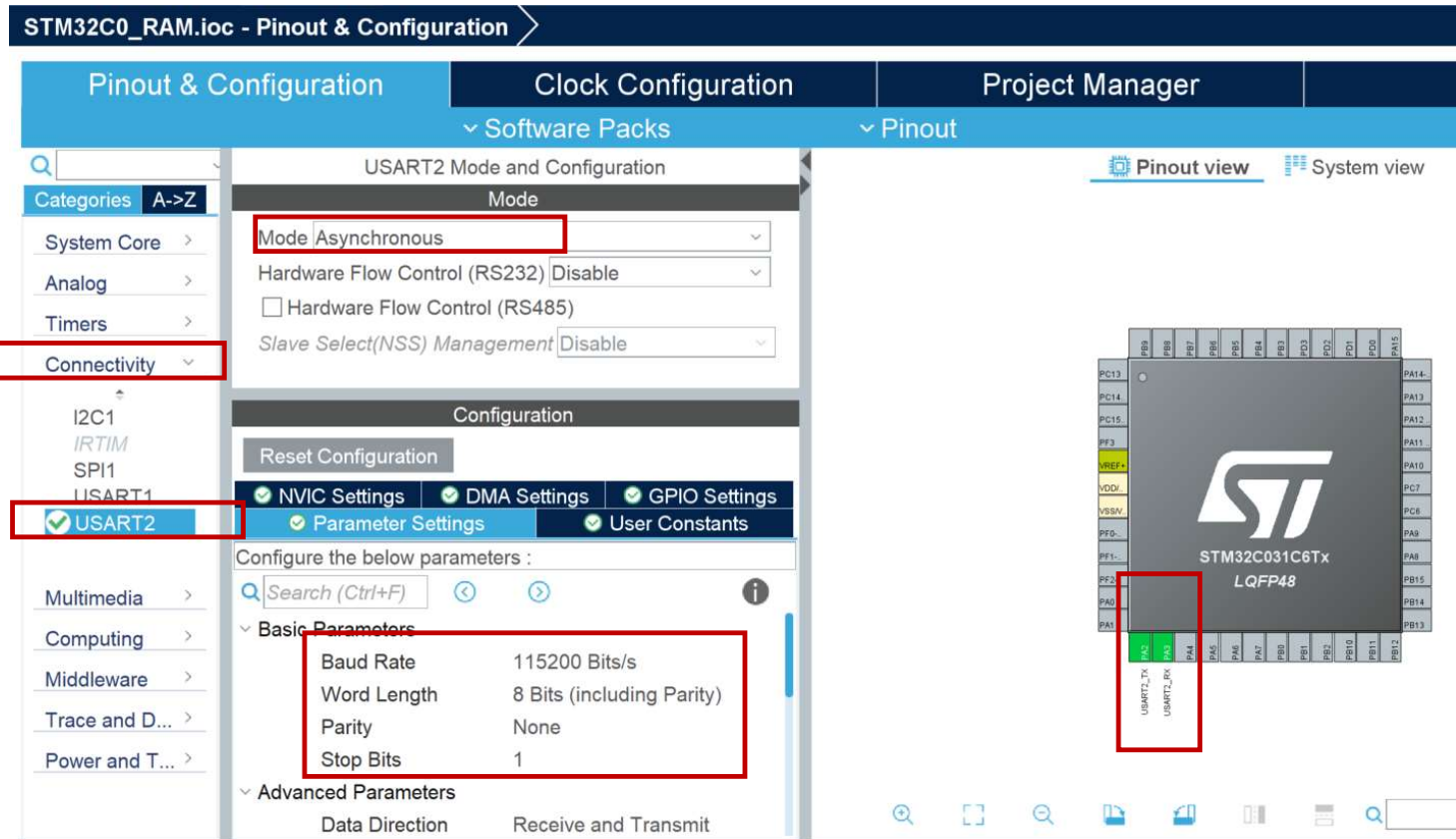
- We will create an application
- We will move part of the code into RAM

Target of HandsOn - Application



STM32C0 UART – UART configuration

- Connectivity
- USART2
- Select Mode:
Asynchronous
- *Basic Parameters*
115200 Bits/s
8N1



STM32C0 UART – UART configuration

- In Configuration in bookmark NVIC Settings.

✓ USART2 interrupt

STM32C0_RAM.ioc - Pinout & Configuration

Pinout & Configuration | Clock Configuration | Project Manager

Software Packs | Pinout

Pinout view | System view

Categories A->Z

- System Core >
- Analog >
- Timers >
- Connectivity >
 - I2C1
 - IRTIM
 - SPI1
 - USART1
 - ✓ USART2
- Multimedia >
- Computing >
- Middleware >
- Trace and D... >
- Power and T... >

USART2 Mode and Configuration

Mode

Mode Asynchronous

Hardware Flow Control (RS232) Disable

☐ Hardware Flow Control (RS485)

Slave Select(NSS) Management Disable

Configuration

Reset Configuration

✓ NVIC Settings | DMA Settings | GPIO Settings

Parameter Settings | User Constants

NVIC Interrupt Table | Enabled | Preemption Priority

USART2 interrupt | ☒ | 0

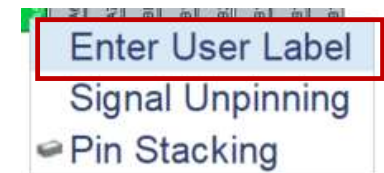
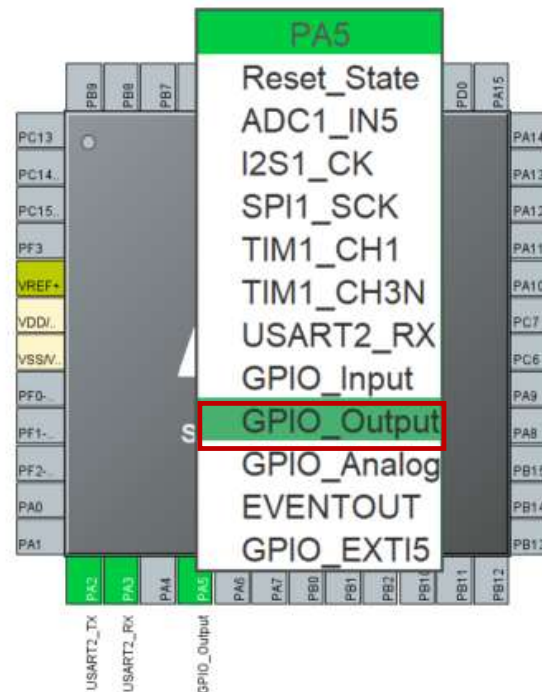


STM32C031C6Tx LQFP48

STM32C0 UART – LED pin configuration

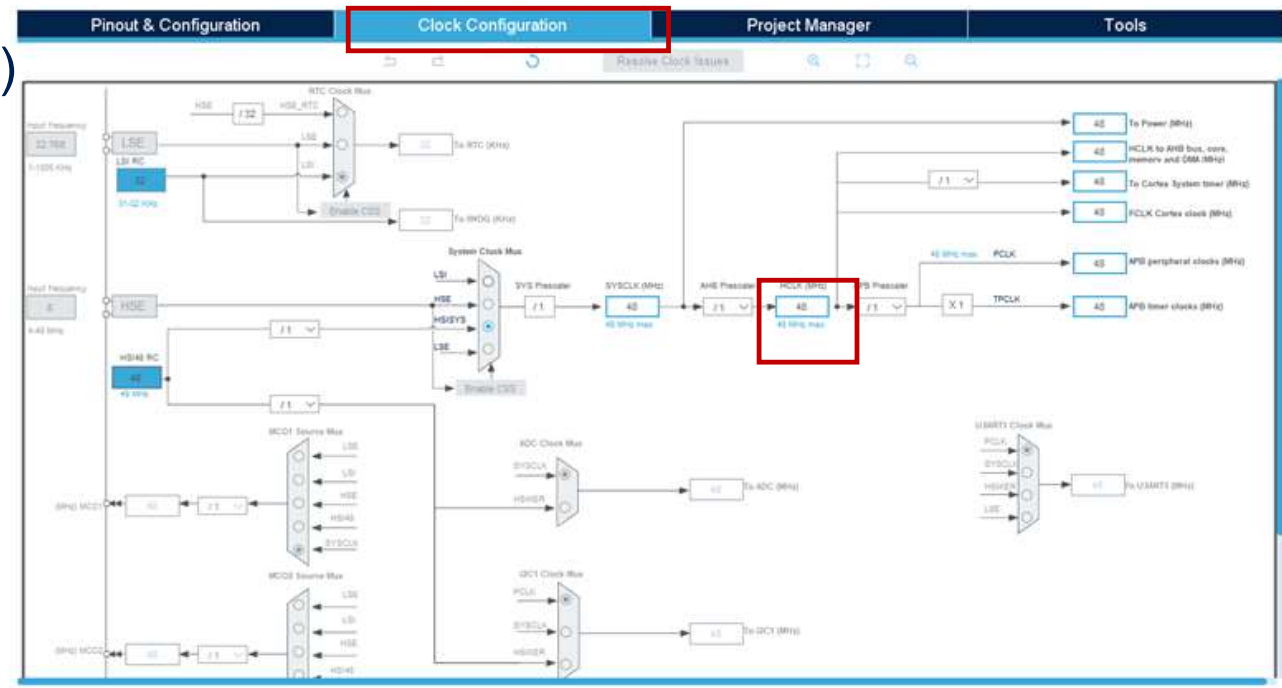
- Set PA5 as output – LED 
- Left Click on PA5 and select as GPIO_Output

- For naming right click
 - Enter User Label
- → Name : **LED**



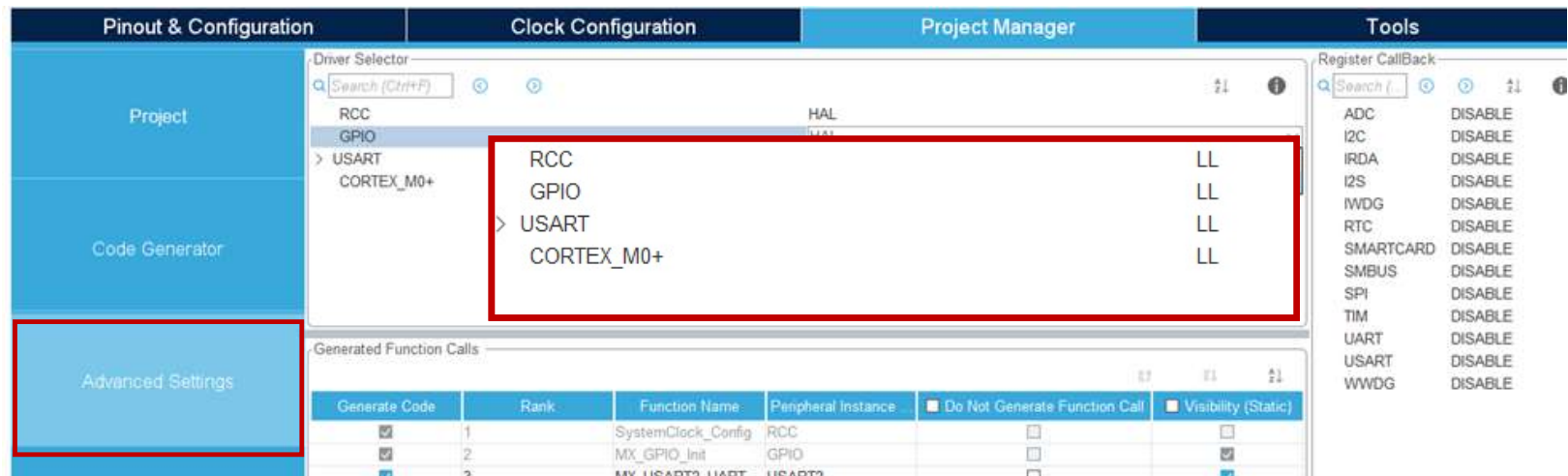
STM32C0 UART – Clock configuration

- Switch to Clock Configuration
- Set HCLK to 48MHz (Enter)



STM32C0 UART – LL library

- Switch to Project Manager → Advanced Setup
- For ALL peripherals change HAL to LL (RCC,GPIO, USART, CORTEX_M0+)





STM32C0 UART – Template Proposals



If you need show Template Proposals, you have to use **Ctrl+Space**

```
98 LL_USART
99 /* USER C
100
101 /* Infini
102 /* USER C
103 while (1)
104 {
105 /* USER
```

- LL_USART_ClearFlag_CM(USART_TypeDef * USARTx) : void
- LL_USART_ClearFlag_EOB(USART_TypeDef * USARTx) : void
- LL_USART_ClearFlag_FE(USART_TypeDef * USARTx) : void
- LL_USART_ClearFlag_IDLE(USART_TypeDef * USARTx) : void
- LL_USART_ClearFlag_LBD(USART_TypeDef * USARTx) : void
- LL_USART_ClearFlag_NE(USART_TypeDef * USARTx) : void
- LL_USART_ClearFlag_ORE(USART_TypeDef * USARTx) : void
- LL_USART_ClearFlag_PE(USART_TypeDef * USARTx) : void
- LL_USART_ClearFlag_RTO(USART_TypeDef * USARTx) : void
- LL_USART_ClearFlag_TC(USART_TypeDef * USARTx) : void
- LL_USART_ClearFlag_TCBGT(USART_TypeDef * USARTx) : void
- LL_USART_ClearFlag_TXFE(USART_TypeDef * USARTx) : void

Press 'Ctrl+Space' to show Template Proposals



STM32C0 UART – code in main.c

- 1st step – Enable interrupts for UART (Receive and Error)
place it **main.c** section `/* USER CODE BEGIN 2 */`

```
LL_USART_EnableIT_RXNE(USART2);  
LL_USART_EnableIT_ERROR(USART2);
```

```
*main.c X  
83  /* Configure the system clock */  
84  SystemClock_Config();  
85  
86  /* USER CODE BEGIN SysInit */  
87  
88  /* USER CODE END SysInit */  
89  
90  /* Initialize all configured peripherals */  
91  MX_GPIO_Init();  
92  MX_USART2_UART_Init();  
93  /* USER CODE BEGIN 2 */  
94  /* Enable RXNE and Error interrupts */  
95  LL_USART_EnableIT_RXNE(USART2);  
96  LL_USART_EnableIT_ERROR(USART2);  
97  /* USER CODE END 2 */  
98  
99  /* Infinite loop */  
100 /* USER CODE BEGIN WHILE */  
101 while (1)
```



STM32C0 UART – code in main.h

- 2nd step – Define callback function

In **main.h** section → Exported functions prototypes, part
/* USER CODE BEGIN EFP */

```
/* USER CODE BEGIN EFP */
```

```
void USART_CharReception_Callback(void);
```

```
/* USER CODE END EFP */
```

```
65  /* Exported functions prototypes -----  
66  void Error_Handler(void);  
67  
68  /* USER CODE BEGIN EFP */  
69  void USART_CharReception_Callback(void);  
70  /* USER CODE END EFP */  
71
```



STM32C0 UART – callback

- 3rd step – Create service routine in callback

In `main.c` section `/* USER CODE BEGIN 4 */`

Save received character and transmit it back ,check received character and compare it with characters 'e', 'E' and if match toggle with LED pin (PA5)

```
void USART_CharReception_Callback(void)
{
    uint8_t received_char;
    received_char = LL_USART_ReceiveData8(USART2);
    if ((received_char == 'E') || (received_char == 'e')) {
        LL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
    }
    LL_USART_TransmitData8(USART2, received_char);
}
```



STM32C0 UART – IRQ Handler

- 4th step – Call service routine in the IRQ Handler

In file **stm32c0xx_it.c** find IRQ Handler for UART2

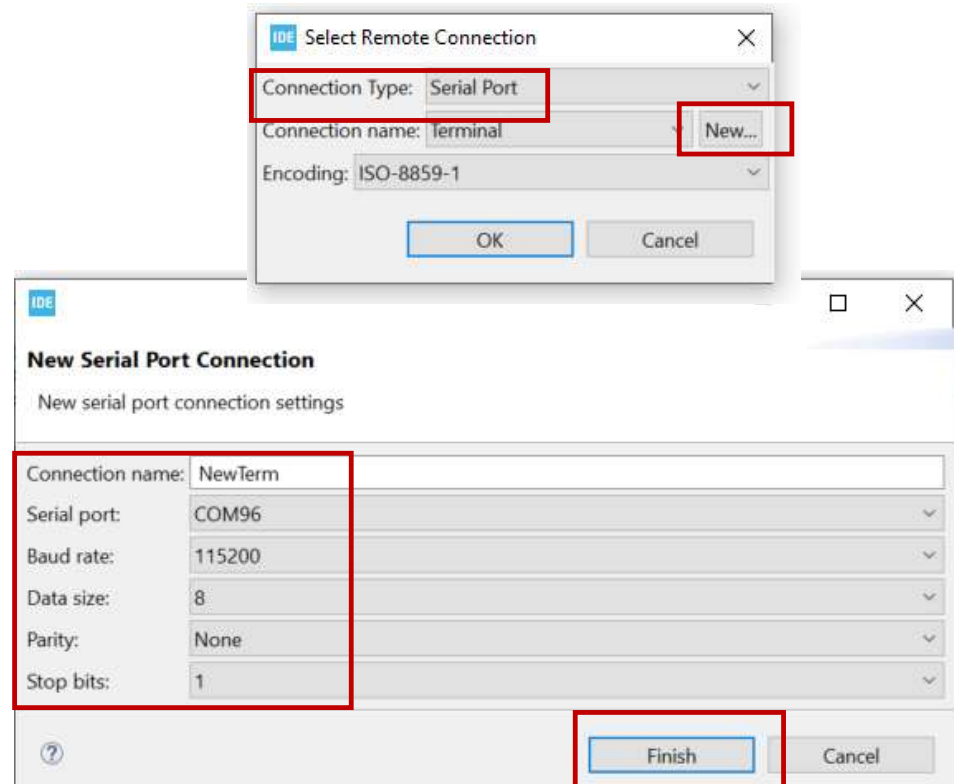
```
void USART2_IRQHandler(void)
{
    /* USER CODE BEGIN USART2_IRQn 0 */
    USART_CharReception_Callback();
    /* USER CODE END USART2_IRQn 0 */
    /* USER CODE BEGIN USART2_IRQn 1 */

    /* USER CODE END USART2_IRQn 1 */
}
```



STM32C0 UART – Terminal

- Open a Terminal on your PC (use your favorite Terminal software)
- Set COM port
- Baud rate : 115200
- Data bits : 8
- Stop bits : 1
- Parity : None





STM32C0 UART – Testing of code

- Write some messages in the Terminal
- By the character **e** or **E** you can turn ON/OFF the LED



The screenshot shows a debugger interface with a 'Console' tab selected. The terminal output consists of a series of lines of 'e' characters, indicating that the LED is turned on. The window title bar shows 'new (CONNECTED)'.

Execution code from RAM

- ✓ Faster than executing from FLASH
 - FLASH 1 wait state
 - RAM **0 wait states**
- ✓ Useful in bootloaders/codes where is need to reprogram FLASH during running of a code
- ✓ less power consumption
- ⊗ Space in FLASH – You have to store in FLASH and then copy to RAM or use external loader

Execution code from RAM

- Where is what stored are defined in **Linker file** or **Linker script**
- Definitions and allocations are made in chronological sequence
- Each IDE has different semantics and process how to set it
- Will do modifications in linker file for **GNU Compiler Collection**
- In cases where vector table is placed in RAM, do not forget to
change VTOR register

Structure of linker file



How to allocate specific variables, function, etc. ?

- defined by specific name. For example:
 - **data** – initialized variables
 - **bss** – non-initialized variables, by compiler initialized to 0
 - **text** – code
 - **rodata** – read only data like a constants, in code **const**
 - ***** – it defines all file/function
 - It is possible to allocate specific function by : ***file.o.text.UserCode**



Goal of the Lab

- Place to RAM some part of code
- Which part of code will be placed in RAM?
 - Drivers – contains LL_drivers
 - Initialization of peripherals
 - USART_CharReception_Callback



Memory occupancy

- Before modification we can check how the memory footprint looks
- **In Build Analyzer** (right down part in STM32CubeIDE) you can see **Memory regions** where is **Usage** of RAM,FLASH in KB and %

Build Analyzer x Static Stack Analyzer Search Type Hierarchy Debug Console

STM32C0_RAM.elf - /STM32C0_RAM/Debug - Nov 21, 2022, 3:04:59 PM

Memory Regions Memory Details

Region	Start address	End address	Size	Free	Used	Usage (%)
RAM	0x20000000	0x20003000	12 KB	10.47 KB	1.53 KB	12.76%
FLASH	0x08000000	0x08008000	32 KB	26.46 KB	5.54 KB	17.31%

▼ RAM	0x20000000		12 KB
> .data	0x20000000	0x080000c0	4 B
.bss	0x20000004		28 B
._user_heap_stack	0x20000020		1,5 KB



Modification of linker file

- In the project open **STM32C031C6TX_FLASH.Id**

The screenshot shows the STM32CubeIDE environment. In the Project Explorer on the left, the file **STM32C031C6TX_FLASH.Id** is selected and highlighted with a red box. The main editor window on the right displays the content of this linker script file. The script is a linker script for the STM32C031C6Tx device, defining memory banks (RAM and FLASH) and their properties.

```
1 /*
2 ****
3 **
4 ** @file      : LinkerScript.ld
5 **
6 ** @author    : Auto-generated by STM32CubeIDE
7 **
8 ** @brief     : Linker script for STM32C031C6Tx Device from STM32C0 series
9 **              32Kbytes FLASH
10 **             12Kbytes RAM
11 **
12 **           Set heap size, stack size and stack location according
13 **           to application requirements.
14 **
15 **           Set memory bank area and size if external memory is used
16 **
17 ** Target    : STMicroelectronics STM32
18 **
19 ** Distribution: The file is distributed as is, without any warranty
20 **               of any kind.
21 **
22 ****
23 ** @attention
24 **
25 ** Copyright (c) 2022 STMicroelectronics.
26 ** All rights reserved.
27 **
28 ** This software is licensed under terms that can be found in the LICENSE file
29 ** in the root directory of this software component.
30 ** If no LICENSE file comes with this software, it is provided AS-IS.
31 **
32 ****
33 */
34
35 /* Entry Point */
36 ENTRY(Reset_Handler)
37
38 /* Highest address of the user mode stack */
39 _estack = ORIGIN(RAM) + LENGTH(RAM); /* end of "RAM" Ram type memory */
40
41 _Min_Heap_Size = 0x200; /* required amount of heap */
42 _Min_Stack_Size = 0x400; /* required amount of stack */
43
44 /* Memories definition */
45 MEMORY
46 {
47   RAM      (xrw)  : ORIGIN = 0x20000000, LENGTH = 12K
48   FLASH    (rx)   : ORIGIN = 0x80000000, LENGTH = 32K
49 }
```



Allocations in chronological sequences

- **Remove** part for data section into RAM line **131-147**
- **Place** it before other data into FLASH line **61**

Modification of linker file

```
130
131 /* Used by the startup to initialize data */
132 _sdata = LOADADDR(.data);
133
134 /* Initialized data sections into "RAM" Ram type memory */
135 .data :
136 {
137     . = ALIGN(4);
138     _sdata = .;          /* create a global symbol at data start */
139     *(.data)             /* .data sections */
140     *(.data*)            /* .data* sections */
141     *(.RamFunc)          /* .RamFunc sections */
142     *(.RamFunc*)         /* .RamFunc* sections */
143
144     . = ALIGN(4);
145     _edata = .;          /* define a global symbol at data end */
146
147 } >RAM AT> FLASH
148
61 /* Used by the startup to initialize data */
62 _sdata = LOADADDR(.data);
63
64 /* Initialized data sections into "RAM" Ram type memory */
65 .data :
66 {
67     . = ALIGN(4);
68     _sdata = .;          /* create a global symbol at data start */
69     *(.data)             /* .data sections */
70     *(.data*)            /* .data* sections */
71     *(.RamFunc)          /* .RamFunc sections */
72     *(.RamFunc*)         /* .RamFunc* sections */
73
74     . = ALIGN(4);
75     _edata = .;          /* define a global symbol at data end */
76
77 } >RAM AT> FLASH
```



Linker file LL drivers

- After `*(.RamFunc*)` line **73** add define path for LL drivers

`*/Drivers/*(.text*)`

```
71  *(.RamFunc)          /* .RamFunc sections */
72  *(.RamFunc*)         /* .RamFunc* sections */
73  */Drivers/*(.text*)
74  . = ALIGN(4);
75
```



Linker file Other initialization

- After `*/Drivers/*(.text*)` define for initialization - **line 74**

```
*main.o(.text.MX_GPIO_Init)
```

```
*main.o(.text.MX_USART2_UART_Init)
```

```
*main.o(.text.USART_CharReception_Callback)
```

```
73      */DRIVERS/*(.text*)
74      *main.o(.text.MX_GPIO_Init)
75      *main.o(.text.MX_USART2_UART_Init)
76      *main.o(.text.USART_CharReception_Callback)
77      - ATCM(4).
```



HandsOn - debug

- Place breakpoint on line **154** and **245** in **main.c** (by double click next to number – left)
also on line **149** in **stm32c0xx_it.c**

```
241 /* USER CODE BEGIN 4 */
242 void USART_CharReception_Callback(void)
243 {
244     int32_t received_char;
245     received_char = LL_USART_ReceiveData8(USART2);
246     if ((received_char == 'E') || (received_char == 'e'))
247     {
248         LL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
249     }
250     LL_USART_TransmitData8(USART2, received_char);
251 }
252 /* USER CODE END 4 */
```

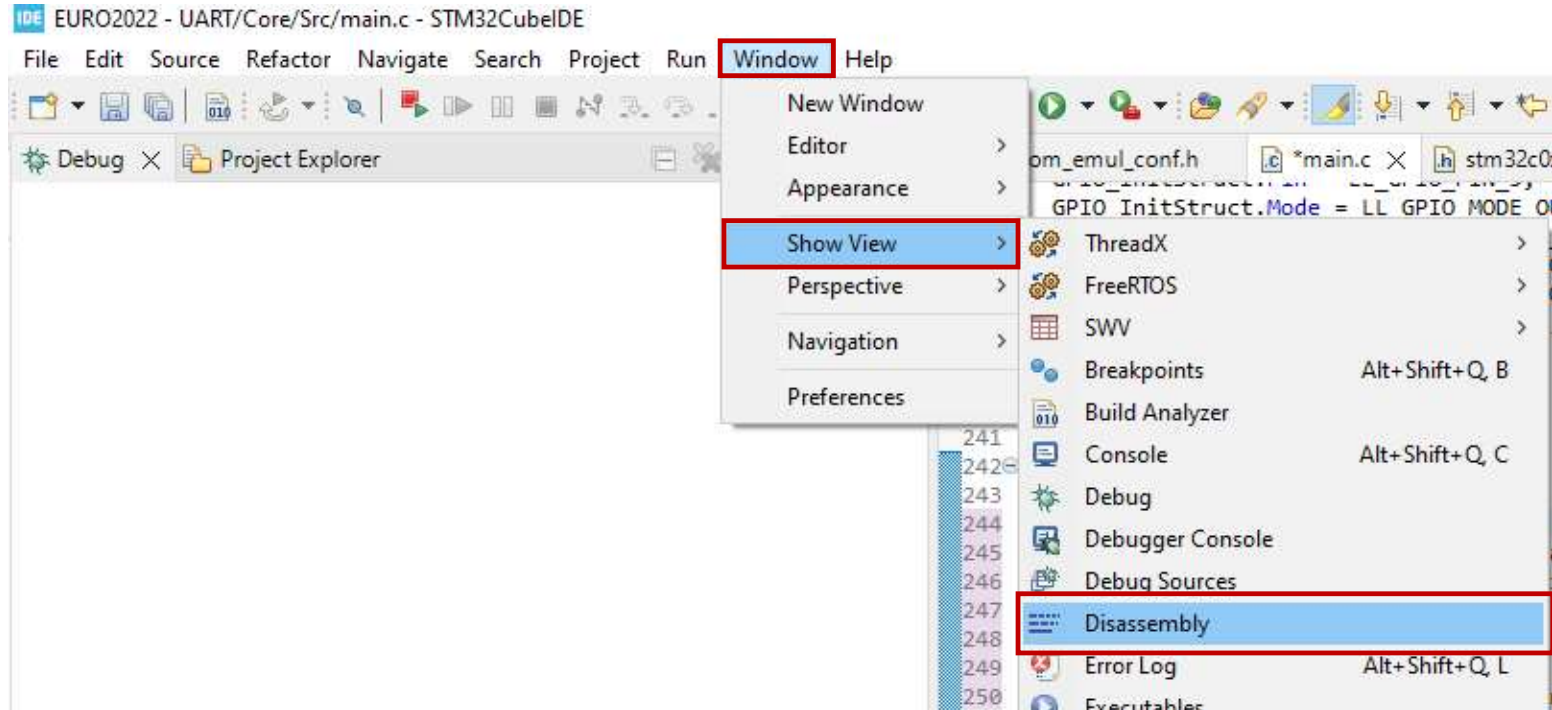
```
147 static void MX_USART2_UART_Init(void)
148 {
149     /* USER CODE BEGIN USART2_Init 0 */
150
151     /* USER CODE END USART2_Init 0 */
152
153     LL_USART_InitTypeDef USART_InitStruct = {0};
154
155     LL_GPIO_InitTypeDef GPIO_InitStruct = {0};
156
157     /* Peripheral clock enable */
158     LL_APB1_GRP1_EnableClock(LL_APB1_GRP1_PERIPH_USART2);
159
160     LL_IOP_GRP1_EnableClock(LL_IOP_GRP1_PERIPH_GPIOA);
161
162     /**USART2 GPIO Configuration
163     PA2 -----> USART2_TX
164     PA3 -----> USART2_RX
165     ...
166     ...
```

```
146 void USART2_IRQHandler(void)
147 {
148     /* USER CODE BEGIN USART2_IRQn 0 */
149     USART_CharReception_Callback();
150     /* USER CODE END USART2_IRQn 0 */
151     /* USER CODE BEGIN USART2_IRQn 1 */
152
153     /* USER CODE END USART2_IRQn 1 */
154 }
155
```



HandsOn - debug

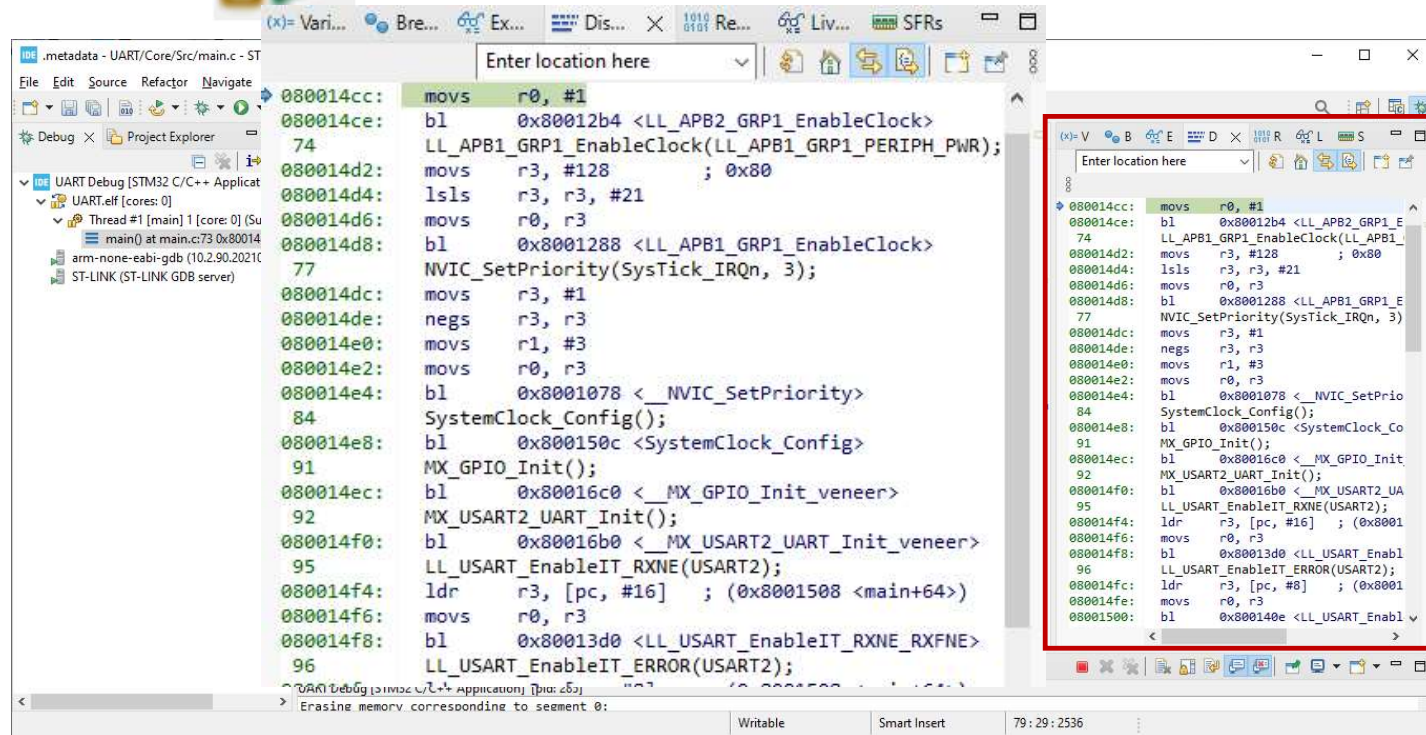
- Click on disassembly window
Window → Show View → Disassembly





HandsOn - debug

- You can see code start in FLASH
- Click on RUN 

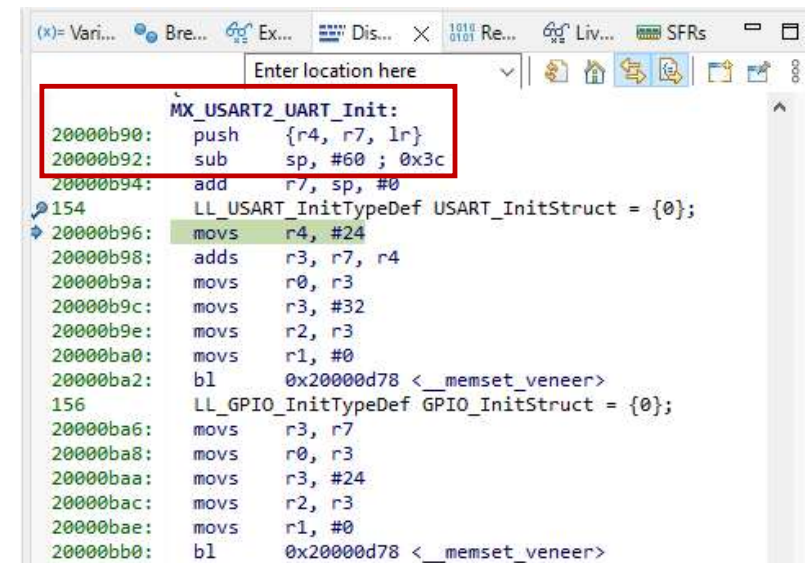


The screenshot displays the ST-Link GDB server interface, showing assembly code and memory addresses. The code is organized into two columns, with the left column showing the current state and the right column showing the state after a run command. The code includes instructions like `movs r0, #1`, `bl 0x80012b4 <LL_APB2_GRP1_EnableClock>`, `LL_APB1_GRP1_EnableClock(LL_APB1_GRP1_PERIPH_PWR);`, `movs r3, #128`, `lsls r3, r3, #21`, `movs r0, r3`, `bl 0x8001288 <LL_APB1_GRP1_EnableClock>`, `NVIC_SetPriority(SysTick_IRQn, 3);`, `movs r3, #1`, `negs r3, r3`, `movs r1, #3`, `movs r0, r3`, `bl 0x8001078 <__NVIC_SetPriority>`, `SystemClock_Config();`, `bl 0x800150c <SystemClock_Config>`, `MX_GPIO_Init();`, `bl 0x80016c0 <__MX_GPIO_Init_veneer>`, `MX_USART2_UART_Init();`, `bl 0x80016b0 <__MX_USART2_UART_Init_veneer>`, `LL_USART_EnableIT_RXNE(USART2);`, `ldr r3, [pc, #16] ; (0x8001508 <main+64>)`, `movs r0, r3`, `bl 0x80013d0 <LL_USART_EnableIT_RXNE_RXFNE>`, `LL_USART_EnableIT_ERROR(USART2);`, `ldr r3, [pc, #8] ; (0x80014fe)`, `movs r0, r3`, and `bl 0x800140e <LL_USART_EnableIT_ERROR>`. The right column shows the same code with the `bl` instructions highlighted in red, indicating they are the next instructions to be executed.

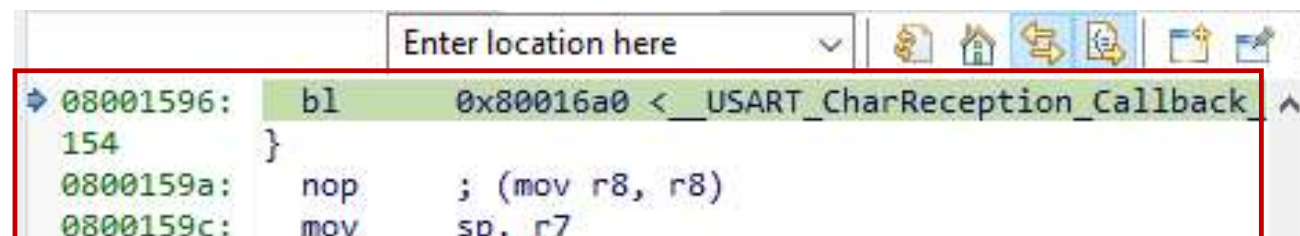


HandsOn - debug

- Code stop on line **154 – UART Init**, you can see address for RAM
- Again, click on RUN 
- **Send some character by Terminal** like in previous HandsOn
- Code stop on line **149 IRQ handler** you can see address for FLASH



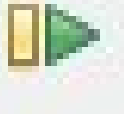
```
(x)= Vari... Bre... Ex... Dis... X 1010 Re... Liv... SFRs
Enter location here
MX_USART2_UART_Init:
20000b90: push    {r4, r7, lr}
20000b92: sub     sp, #60 ; 0x3c
20000b94: add     r7, sp, #0
154 20000b96: movs    r4, #24
20000b98: adds    r3, r7, r4
20000b9a: movs    r0, r3
20000b9c: movs    r3, #32
20000b9e: movs    r2, r3
20000ba0: movs    r1, #0
20000ba2: bl      0x2000d78 <__memset_vener>
156 20000ba6: movs    r3, r7
20000ba8: movs    r0, r3
20000baa: movs    r3, #24
20000bac: movs    r2, r3
20000bae: movs    r1, #0
20000bb0: bl      0x2000d78 <__memset_vener>
```

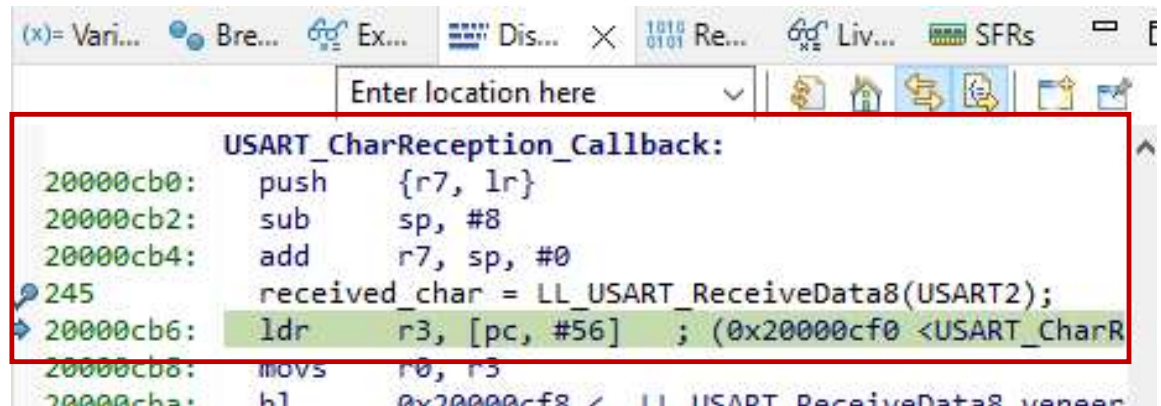


```
Enter location here
08001596: bl      0x80016a0 <__USART_CharReception_Callback>
154      }
0800159a: nop     ; (mov r8, r8)
0800159c: mov     sp, r7
```



HandsOn - debug

- Again, click on RUN 
- Last stop is on line **245 – UART callback**, also you can see address for RAM



```
(x)= Vari... Bre... Ex... Dis... Re... Liv... SFRs
Enter location here
USART_CharReception_Callback:
20000cb0: push    {r7, lr}
20000cb2: sub     sp, #8
20000cb4: add     r7, sp, #0
245      received_char = LL_USART_ReceiveData8(USART2);
20000cb6: ldr     r3, [pc, #56] ; (0x20000cf0 <USART_CharR
20000cb8: movs    r0, r3
20000cba: bl      0x20000cf8 < LL_USART_ReceiveData8 verson
```